Project work part 3 - IND320

Malk

Imports and load data

```
import pandas as pd
import matplotlib.pyplot as plt
import requests

import tomllib
import pandas as pd
from entsoe import EntsoePandasClient

import os
import sys
from pathlib import Path

from cassandra.cluster import Cluster
import pyspark
from pyspark.sql import SparkSession
from pyspark.sql.types import (
    StructType, StructField, StringType,
    IntegerType, TimestampType, DoubleType
)
from pymongo import MongoClient, UpdateOne
import scipy.stats as stats
from scipy.fft import dct, idct
import numpy as np

from sklearn.neighbors import LocalOutlierFactor
from matplotlib.lines import Line2D
from statsmodels.tsa.seasonal import STL

#fetching the data

#Base address of the NVE Magasinstatistikk API
BASE_URL = "https://biapi.nve.no/magasinstatistikk"

def fetch_json(endpoint):
    """Call one API endpoint and return the parsed JSON."""
    response = requests.get(
        f"{BASE_URL}{endpoint}",
        headers={"Accept": "application/json"},
        timeout=60,
    )
    response.raise_for_status()
    return response.json()
```

```
df_file =
pd.DataFrame(fetch_json("/api/Magasinstatistikk/HentOffentligData"))
```

Describing the dataset and preprocessing

```
#general information about the data
#size of the dataset
# the first 5 rows of the dataset
#information about the columns of the dataset

print(df_file.shape)
print(df_file.head())
print(df_file.info())
```

```
(14922, 11)
      dato_Id omrType  omrnr  iso_aar  iso_uke  fyllingsgrad
kapasitet_TWh  \
0  1995-09-03      EL      4     1995       35      0.944721
21.079208
1  2020-11-15      EL      1     2020       46      0.974361
6.003264
2  2011-08-28      EL      4     2011       34      0.786499
21.079208
3  1998-07-05      EL      3     1998       27      0.793969
8.920932
4  2011-03-27      EL      4     2011       12      0.300820
21.079208

   fylling_TWh neste_Publiseringsdato  fyllingsgrad_forrige_uke  \
0    19.913979    0001-01-01T00:00:00                  0.936888
1     5.849344    2020-11-25T13:00:00                  0.997406
2    16.578772    0001-01-01T00:00:00                  0.787193
3     7.082947    0001-01-01T00:00:00                  0.733812
4     6.341054    0001-01-01T00:00:00                  0.311122

   endring_fyllingsgrad
0              0.007833
1             -0.023046
2             -0.000694
3              0.060157
4             -0.010301
<class 'pandas.DataFrame'>
RangeIndex: 14922 entries, 0 to 14921
Data columns (total 11 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   dato_Id                   14922 non-null  str
 1   omrType                   14922 non-null  str
 2   omrnr                     14922 non-null  int64
 3   iso_aar                   14922 non-null  int64
```

```
 4   iso_uke                   14922 non-null  int64
 5   fyllingsgrad              14922 non-null  float64
 6   kapasitet_TWh             14922 non-null  float64
 7   fylling_TWh               14922 non-null  float64
 8   neste_Publiseringsdato    14922 non-null  str
 9   fyllingsgrad_forrige_uke  14922 non-null  float64
 10  endring_fyllingsgrad      14922 non-null  float64
dtypes: float64(5), int64(3), str(3)
memory usage: 1.7 MB
None

#describing the dataset in a more statistical sense
#along with number of observations for each column
print(df_file.describe())
print(df_file.isna().sum())

              omrnr       iso_aar       iso_uke  fyllingsgrad
kapasitet_TWh  \
count  14922.000000  14922.000000  14922.000000  14922.000000
14922.000000
mean       2.333333   2010.393245     26.443908      0.618948
29.145860
std        1.490762      9.173142     15.011341      0.214686
22.739068
min        0.000000   1995.000000      1.000000      0.051653
6.003264
25%        1.000000   2002.000000     13.000000      0.456817
17.390587
50%        2.000000   2010.000000     26.000000      0.648094
23.237715
75%        3.000000   2018.000000     39.000000      0.801885
34.043590
max        5.000000   2026.000000     53.000000      1.020072
87.437580

        fylling_TWh  fyllingsgrad_forrige_uke  endring_fyllingsgrad
count  14922.000000              14922.000000          14922.000000
mean      18.142298                  0.618940              0.000008
std       15.962981                  0.214672              0.031334
min        0.310089                  0.051653             -0.242484
25%        7.325947                  0.456817             -0.022360
50%       14.504506                  0.648094             -0.006960
75%       22.203242                  0.801785              0.016274
max       84.518060                  1.020037              0.345632
dato_Id                     0
omrType                     0
omrnr                       0
iso_aar                     0
iso_uke                     0
fyllingsgrad                0
```

```
kapasitet_TWh                0
fylling_TWh                  0
neste_Publiseringsdato       0
fyllingsgrad_forrige_uke     0
endring_fyllingsgrad         0
dtype: int64
```

The dataset looks complete in the sense of no missing values in any of the columns. Stay tuned to find out if this truly is the case. The first registration was made in 1996 (min-value for iso-aar) while the latest registration was made in 2026.

Some of the headers explain themselves, others are not that easy to interpret.

```
df = df_file.copy() #renaming

#renaming the columns of the dataset
df = df.rename(columns={'dato_Id': 'Date', "omrType": "Area_type",
"omrnr": "Area_number",
                        "iso_aar": "ISO_year","iso_uke": "ISO_week",
                        "fyllingsgrad": "Filling_level",
"kapasitet_TWh": "Capacity_TWh",
                        "fylling_TWh": "Filling_TWh",
                        "neste_Publiseringsdato": "Next_publishdate",
                        "fyllingsgrad_forrige_uke":
"Filling_level_prev_week",
                        "endring_fyllingsgrad":
"Change_filling_level"})

#sorting the dataframe by date so the datapoints become sequential
df = df.sort_values("Date")

#checking if the changes worked
df.head()
```

```
            Date Area_type  Area_number  ISO_year  ISO_week
Filling_level  \
3014  1995-01-08        EL            4      1995         1
0.558864
3111  1995-01-08        EL            5      1995         1
0.616448
4579  1995-01-08        EL            3      1995         1
0.602376
7282  1995-01-08        EL            2      1995         1
0.630947
7882  1995-01-08        EL            1      1995         1
0.617574

      Capacity_TWh  Filling_TWh     Next_publishdate
Filling_level_prev_week  \
3014     21.079208    11.780413  0001-01-01T00:00:00
```

```
0.801348
3111     17.390587    10.720388  0001-01-01T00:00:00
0.614288
4579      8.920932     5.373754  0001-01-01T00:00:00
0.596027
7282     34.043590    21.479713  0001-01-01T00:00:00
0.777604
7882      6.003264     3.707457  0001-01-01T00:00:00
0.583705

      Change_filling_level
3014             -0.242484
3111              0.002160
4579              0.006349
7282             -0.146657
7882              0.033869
```

We see that "Next_publishdate" for some measurements are 0001-01-01T00:00:00, which doesnt give any intiuitive sense, but it could be that before that date, the system wasnt digitalised. This date is then the minimum date.

We can observe that the column "Date" is in the format off YYYY-MM-DD, but the type is an object, and the same is for "Next_Publishdate".

```
# converting the date columns to datetime format
for col in ["Date", "Next_publishdate"]:
    df[col] = pd.to_datetime(df[col], format="ISO8601",
errors="coerce")

print(df[["Date", "Next_publishdate"]].dtypes)
```

```
Date                datetime64[us]
Next_publishdate    datetime64[us]
dtype: object
```

```
#checking if the changes were made
df.head()
```

```
           Date Area_type  Area_number  ISO_year  ISO_week
Filling_level  \
3014 1995-01-08        EL            4      1995         1
0.558864
3111 1995-01-08        EL            5      1995         1
0.616448
4579 1995-01-08        EL            3      1995         1
0.602376
7282 1995-01-08        EL            2      1995         1
0.630947
7882 1995-01-08        EL            1      1995         1
0.617574
```

```
      Capacity_TWh  Filling_TWh Next_publishdate
Filling_level_prev_week  \
3014     21.079208    11.780413          1-01-01
0.801348
3111     17.390587    10.720388          1-01-01
0.614288
4579      8.920932     5.373754          1-01-01
0.596027
7282     34.043590    21.479713          1-01-01
0.777604
7882      6.003264     3.707457          1-01-01
0.583705

      Change_filling_level
3014             -0.242484
3111              0.002160
4579              0.006349
7282             -0.146657
7882              0.033869
```

NaT = Not a Time. Happens when there are missing dates within a dataframe. 0001-01-01T00:00:00 is not a date and the value therefore becomes NaT. So even though isna().sum() gave us that there is no missing values, by actually inspecting the dataset we see that there is indeed missing values.

Plotting each column

```
print(df["Area_type"].unique()) #checking unique values in the
Area_type column
print(df.groupby("Area_type")["Area_number"].unique())

<ArrowStringArray>
['EL', 'NO', 'VASS']
Length: 3, dtype: str
Area_type
EL      [4, 5, 3, 2, 1]
NO                  [0]
VASS          [3, 2, 1]
Name: Area_number, dtype: object
```

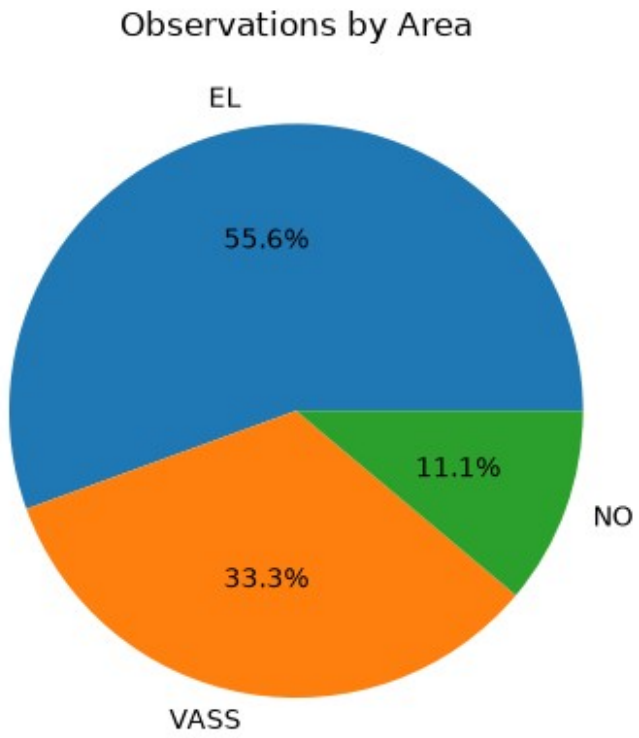
This columns seems to hold information about how the data is categorized in areas, wether it is

- NO: The whole country of Norway
- EL: one of the five electricity price areas
  - EL1
  - EL2
  - EL3
  - EL4
  - EL5

- VASS: river basin area (Vassdragsområde)
  - VASS1
  - VASS2
  - VASS3

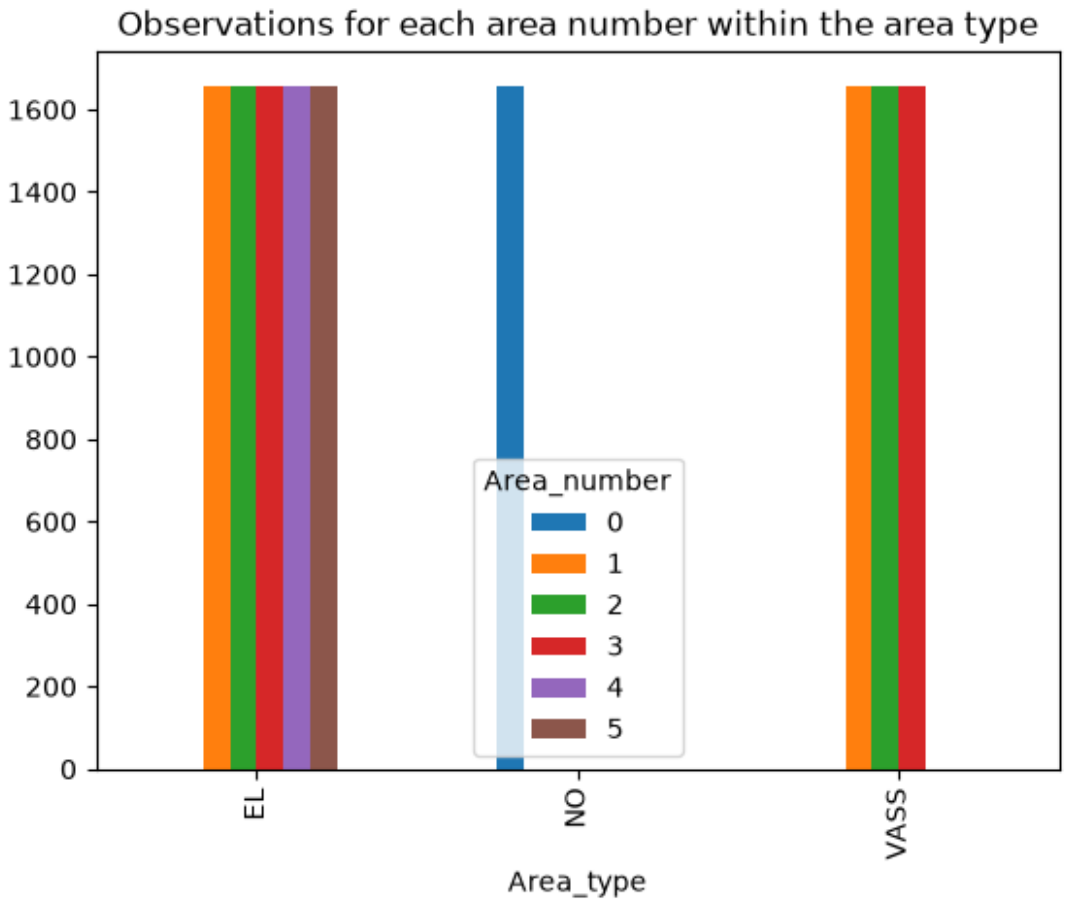
```
# how many observations are made for each area type
df["Area_type"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Observations by Area")
```

```
Text(0.5, 1.0, 'Observations by Area')
```




```
#different area numbers for each area type
df.groupby(["Area_type",
"Area_number"]).size().unstack(fill_value=0).plot(kind="bar")
plt.title("Observations for each area number within the area type")
```

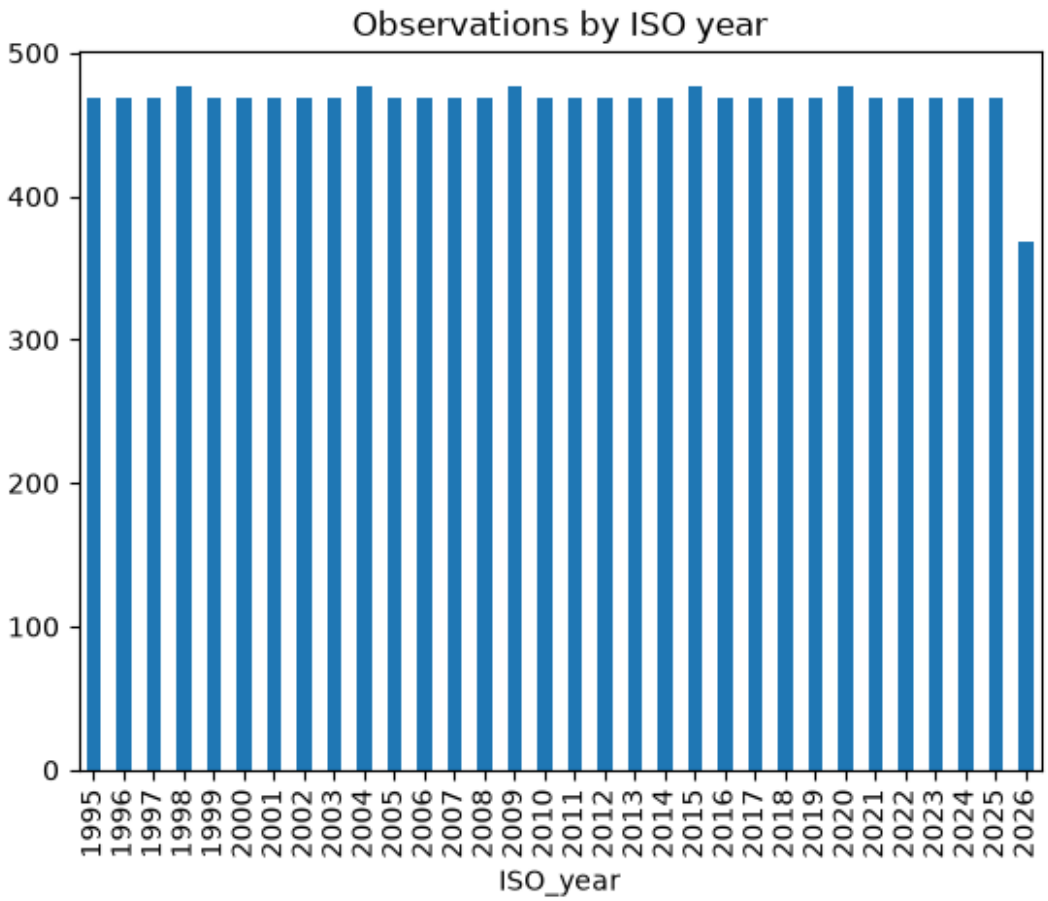
```
Text(0.5, 1.0, 'Observations for each area number within the area
type')
```



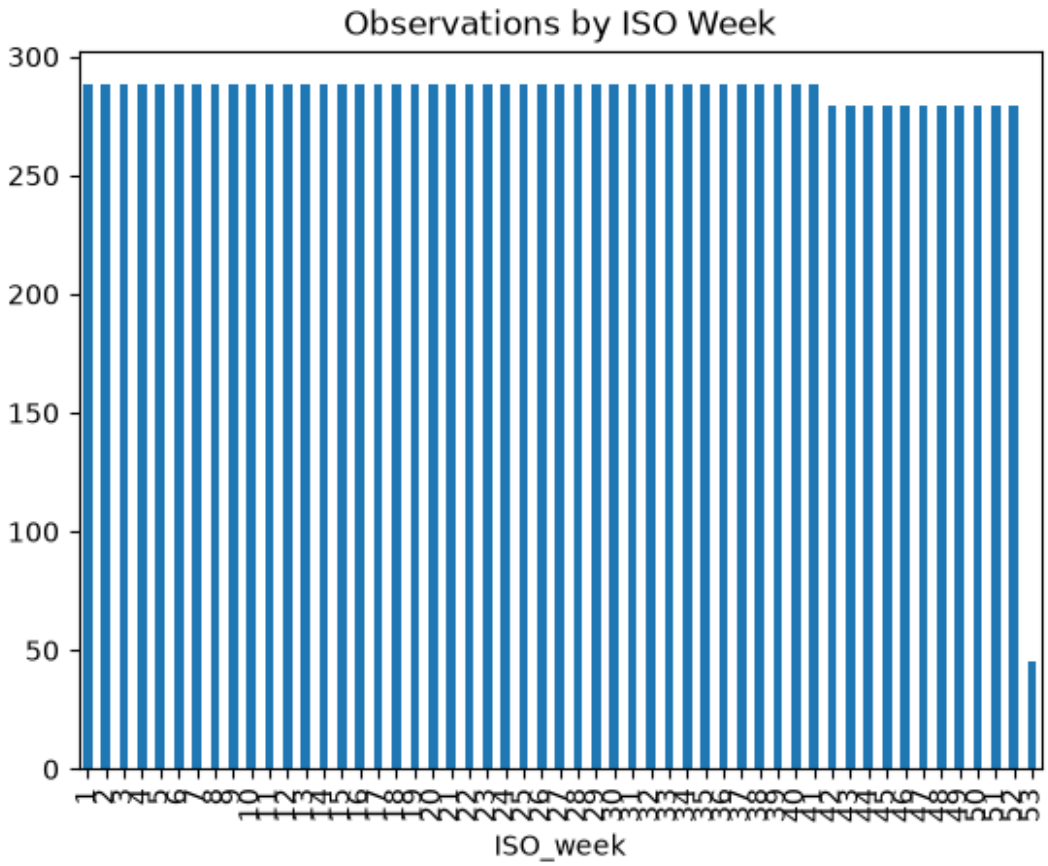
NO has no area number as it covers the whole country, so the default area number is zero. However, each area number is represented in the same amount, this can also seen in the pie chart.

```
#how many observations there are for each ISO year
#ISO years are essentially observations throughout the years, grouped by year
df["ISO_year"].value_counts().sort_index().plot(kind="bar")
plt.title("Observations by ISO year")

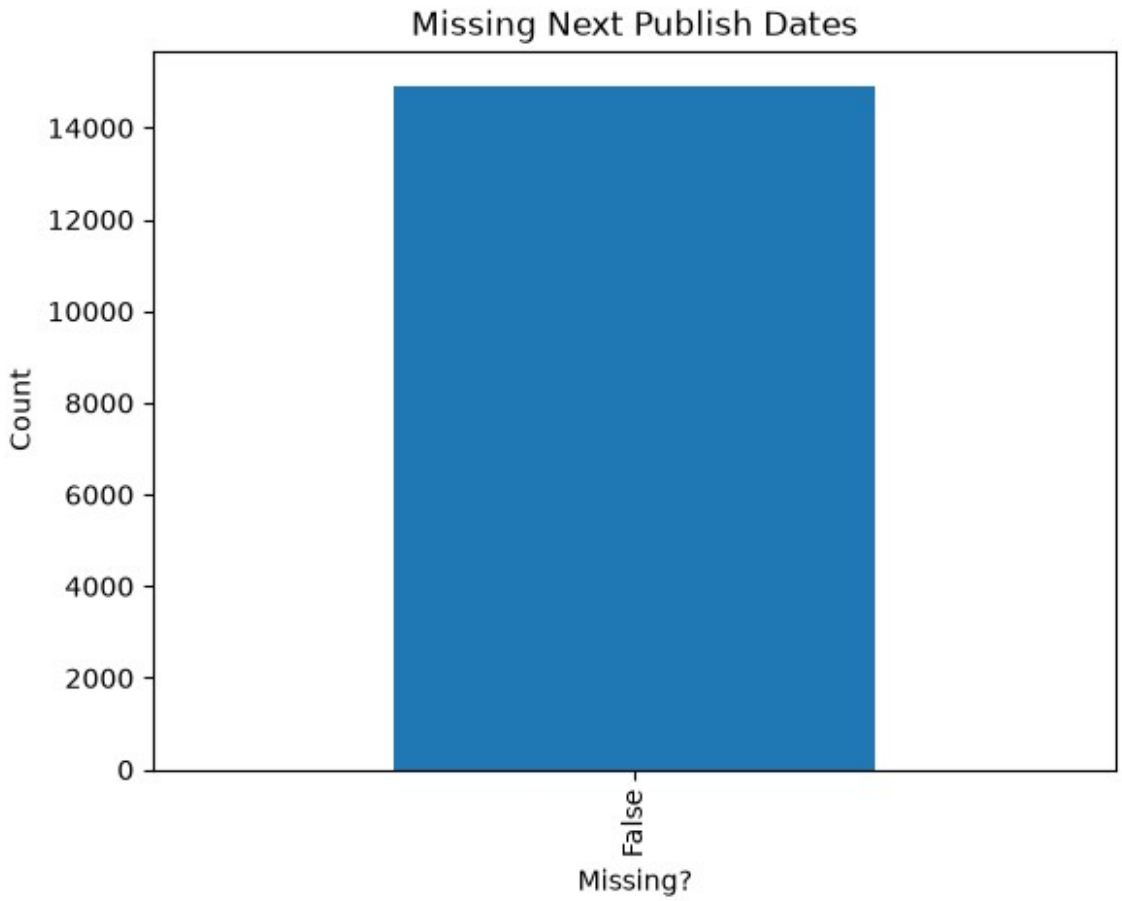
Text(0.5, 1.0, 'Observations by ISO year')
```




```
#how many observations there are for each ISO week
#ISO weeks are essentially obervations throughout the years, grouped
by week number
df["ISO_week"].value_counts().sort_index().plot(kind="bar")
plt.title("Observations by ISO Week")
plt.show()
```




```
#how many of the Next_publishdate is NaT
df["Next_publishdate"].isna().value_counts().plot(kind="bar")
plt.title("Missing Next Publish Dates")
plt.xlabel("Missing?")
plt.ylabel("Count")
plt.show()
```



Even though we showed above that there was no missing values, looking further into the data, we now see that the missing values were replaced by a default date, and then when we transfomed the column to have a proper date-format, the missing values became marked as NaT.

It doesn't make sense to plot the Date-column by itself, so in stead i will group the measurements by month, and then look at when the measurements are taken throughout the seasons of the year.

```
#count the number of measurement for each month of the year
#then assign the months an index from 1-12
monthly_measurements = df["Date"].dt.month.value_counts().sort_index()

#assign the indexed months to their respective season
#sum all the observations for each month within the season
spring_months = monthly_measurements[[3, 4, 5]].sum()
summer_months = monthly_measurements[[6, 7, 8]].sum()
fall_months = monthly_measurements[[9, 10, 11]].sum()
winter_months = monthly_measurements[[12, 1, 2]].sum()

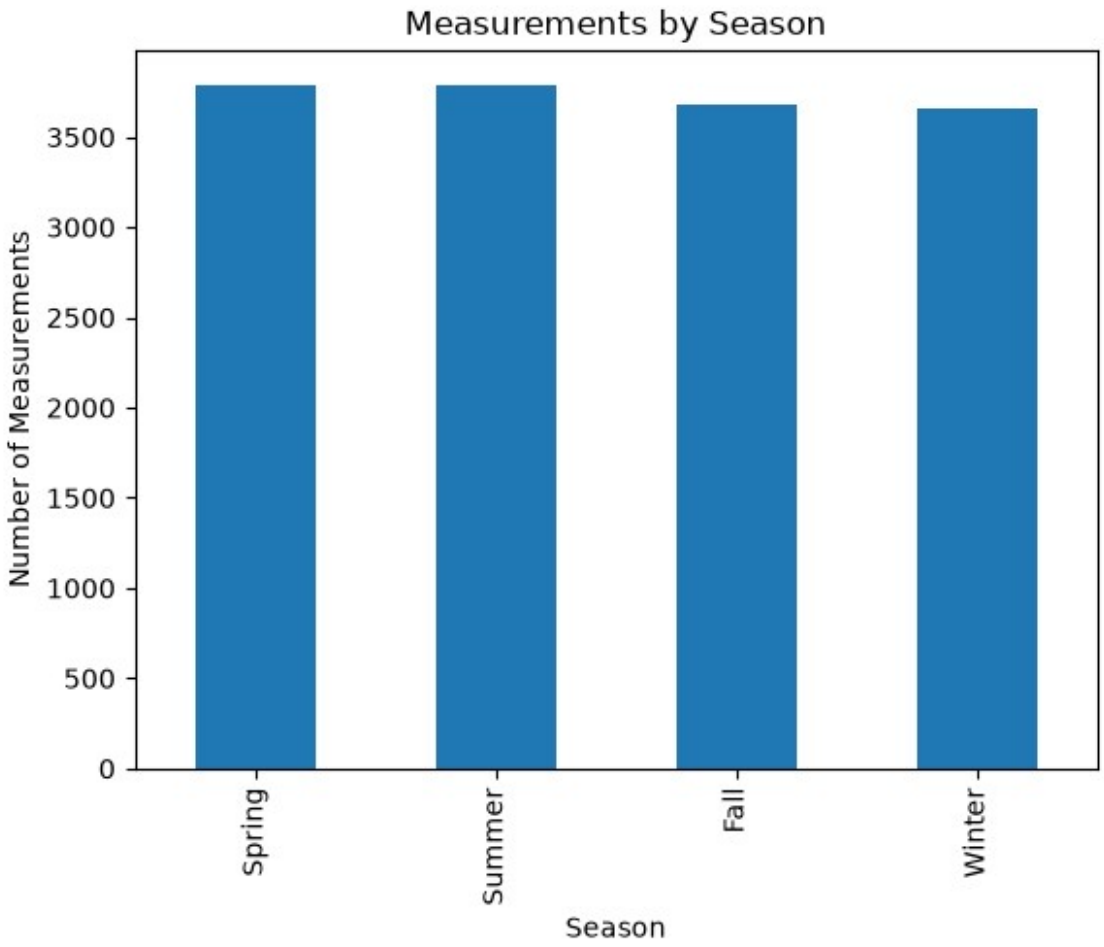
#create a pandas series where each season get its corresponding
#number of measurements
```

```
seasonal_measurements =  pd.Series(
    [spring_months, summer_months, fall_months, winter_months],
    index=["Spring", "Summer", "Fall", "Winter"]
)

seasonal_measurements.plot(kind="bar")

plt.title("Measurements by Season")
plt.xlabel("Season")
plt.ylabel("Number of Measurements")
```

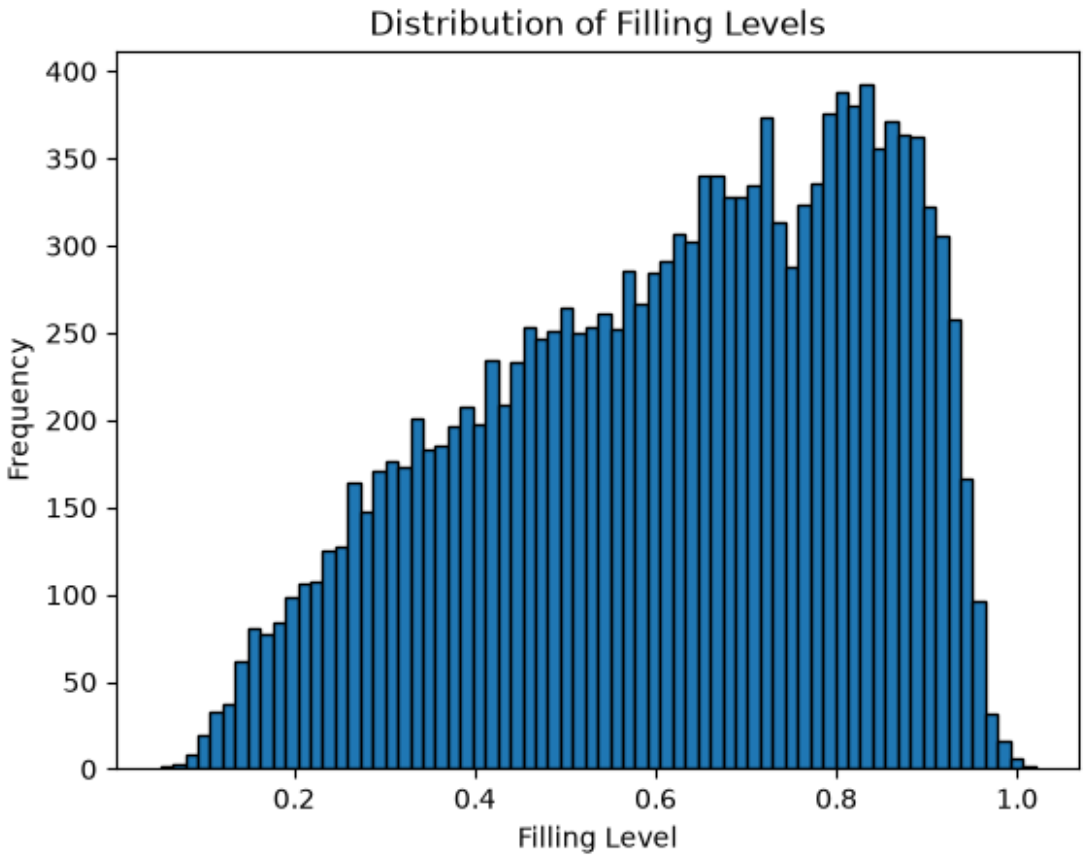
```
Text(0, 0.5, 'Number of Measurements')
```



Judging by this, it looks like there are fewer measurements during the colder months in contrary to the summer and spring months. However, the difference doesnt seem signficant.

```
df["Filling_level"].plot(kind="hist", bins=70, edgecolor="black")
#tried different bin sizes, 70 seems good
plt.title("Distribution of Filling Levels")
plt.xlabel("Filling Level")
```

```
Text(0.5, 0, 'Filling Level')
```

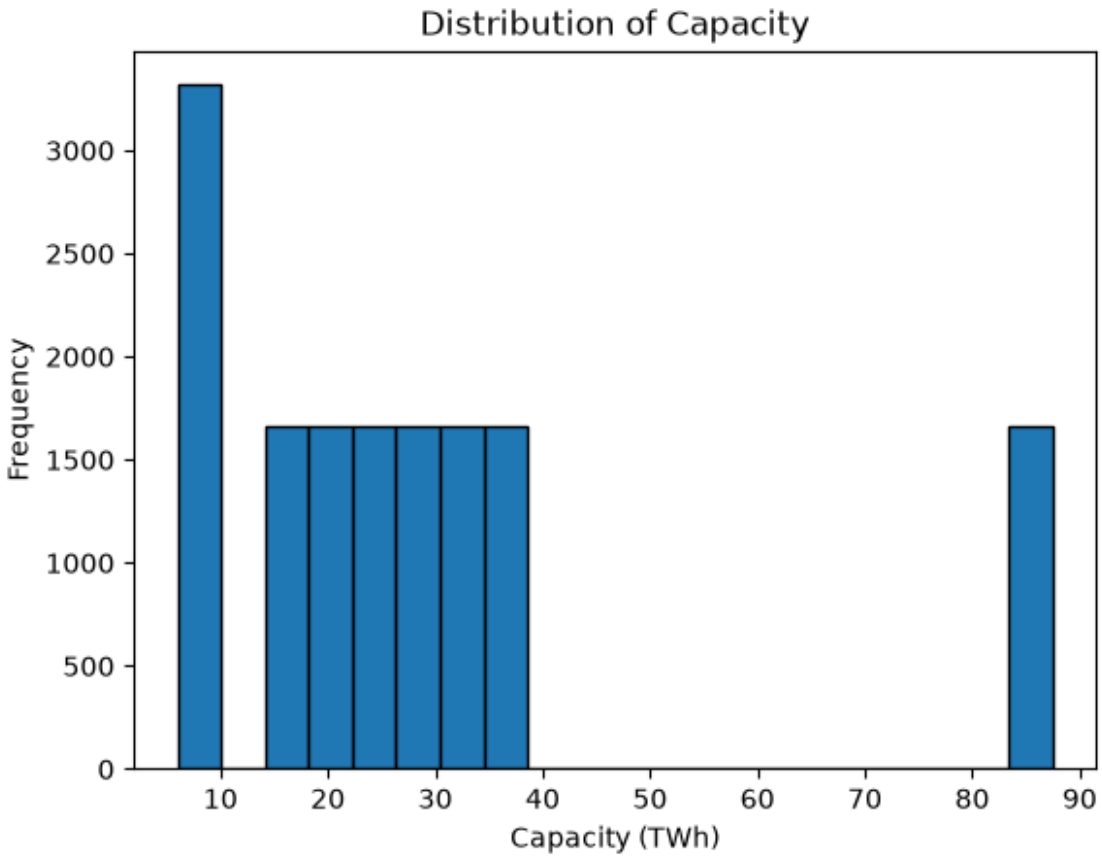


The distribution of the filling level is slightly left-skewed. This means that most reservoirs have relatively high filling levels, but there are some observations with unusually low filling levels.

```
df["Capacity_TWh"].plot(kind="hist", bins=20, edgecolor="black")

plt.title("Distribution of Capacity")
plt.xlabel("Capacity (TWh)")

Text(0.5, 0, 'Capacity (TWh)')
```

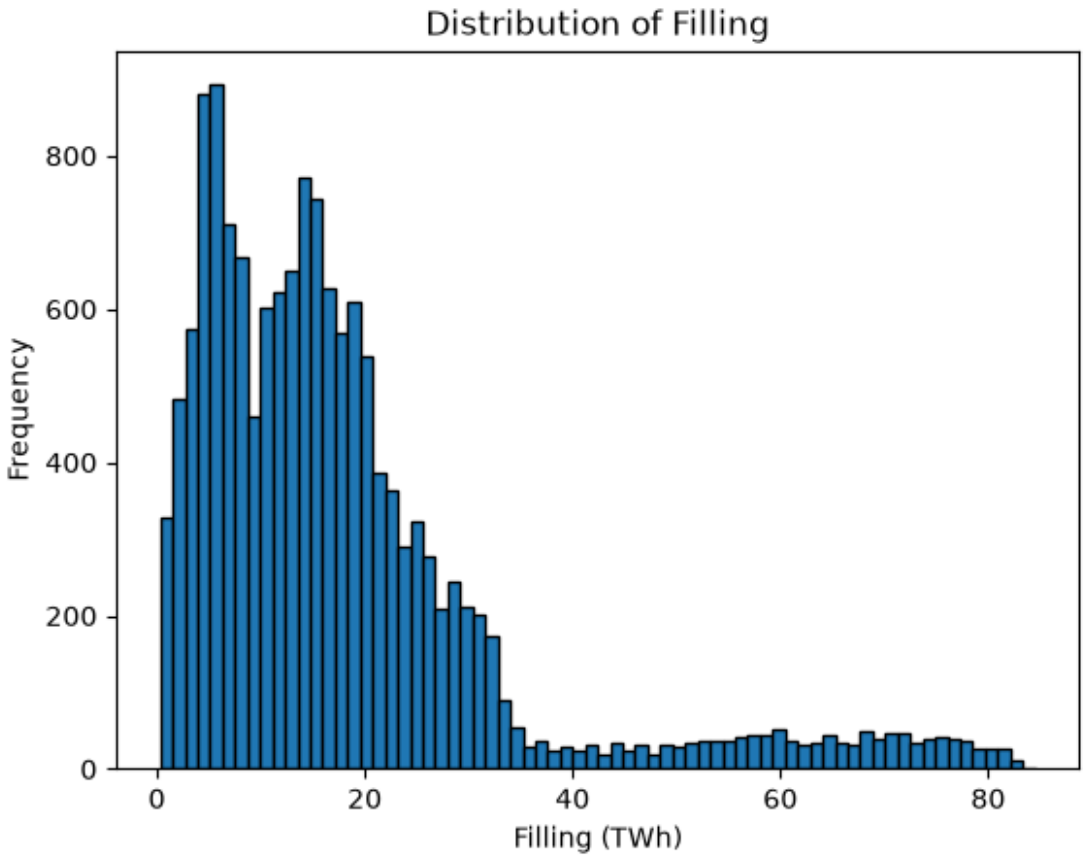


The distribution of capacity shows that most observations have a capacity of around 10 TWh. The remaining observations are mainly distributed between 15 and 40 TWh, with a few observations having a capacity of approximately 85 TWh.

```
df["Filling_TWh"].plot(kind="hist", bins=70, edgecolor="black")

plt.title("Distribution of Filling")
plt.xlabel("Filling (TWh)")

Text(0.5, 0, 'Filling (TWh)')
```

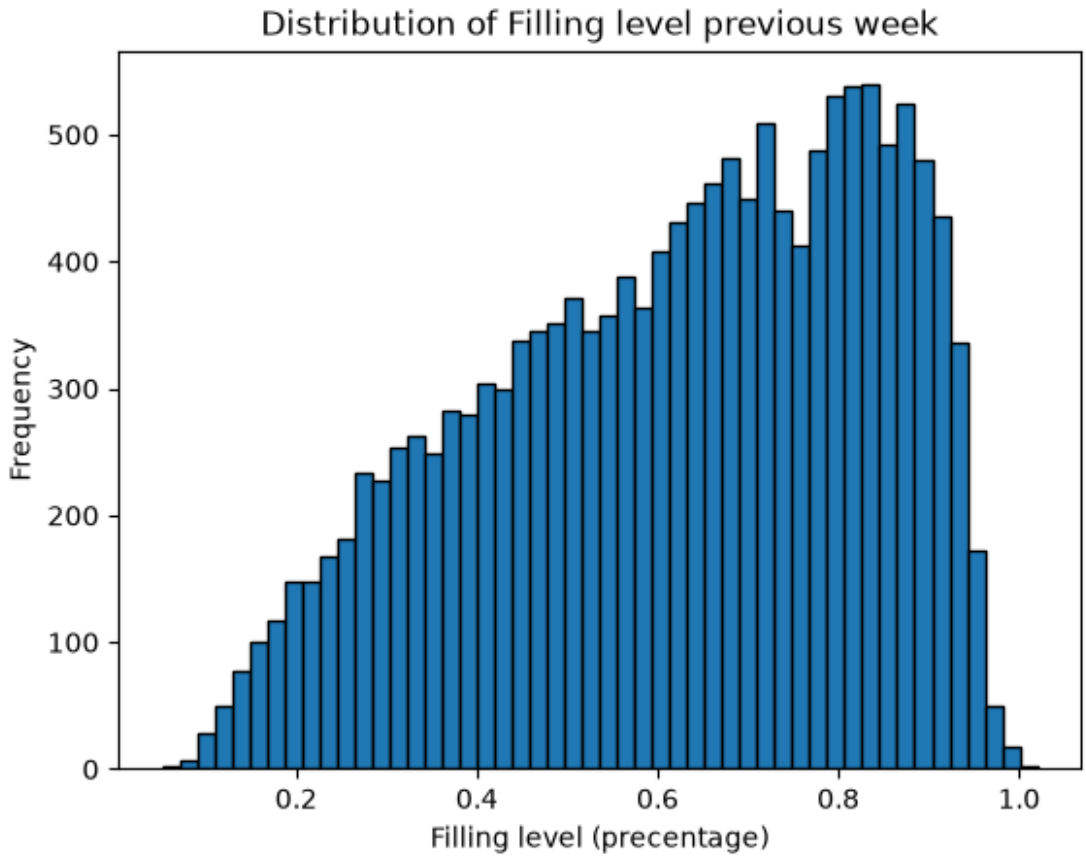


The distribution of Filling TWh is right-skewed. This means that most observations are concentrated at lower values, while a smaller number of observations have higher values, creating a longer tail toward the right.

```
df["Filling_level_prev_week"].plot(kind="hist", bins=50,
edgecolor="black")

plt.title("Distribution of Filling level previous week")
plt.xlabel("Filling level (precentage)")
```

```
Text(0.5, 0, 'Filling level (precentage)')
```

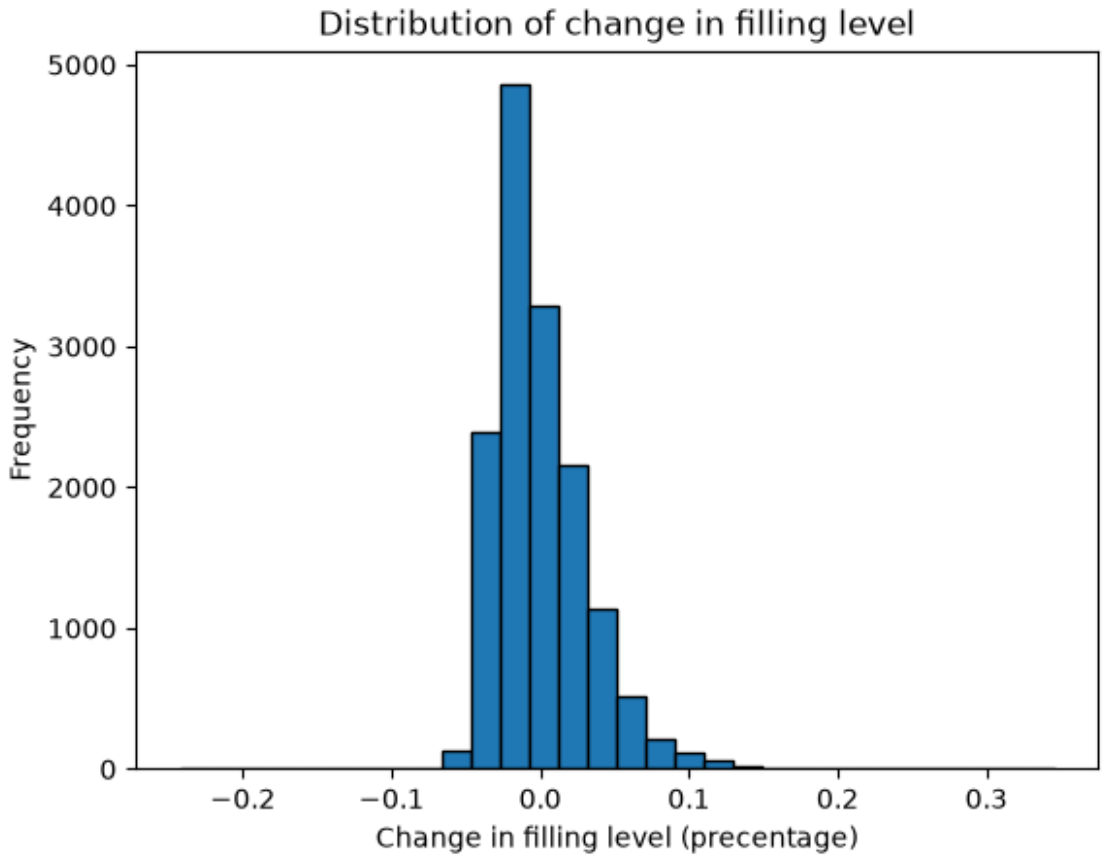


The distributiomn of the filling level for the previous week is slightly left-skewed. Most observations are concentrated toward the higher filling levels, while the observations gradually decrease toward the lower values. However, the skewness is not very pronounced.

```
df["Change_filling_level"].plot(kind="hist", bins=30,
edgecolor="black")

plt.title("Distribution of change in filling level")
plt.xlabel("Change in filling level (precentage)")

Text(0.5, 0, 'Change in filling level (precentage)')
```




```
# counting the number of observations for the change in filling level
below_zero = (df["Change_filling_level"] < 0).sum() #the change is
negative = less filling than last measured
above_zero = (df["Change_filling_level"] > 0).sum() #the change is
positive = more filling than last measured
at_zero = (df["Change_filling_level"] == 0).sum() #the change is zero
= no change from last time

print("Below 0:", below_zero)
print("Above 0:", above_zero)
print("Exactly 0:", at_zero)
```

```
Below 0: 8708
Above 0: 6214
Exactly 0: 0
```

Then we have the distribution of the change in filling level. Most observations are below zero, meaning on the left side, giving a distribution with a tail on right-side, making it right-skewed.

Plotting columns against each other

Plotting the measurements over time, where the datapoints are grouped by area_type and then looking at area_number for each area_type.

```
el_area = df[df["Area_type"] == "EL"].copy() #filtering the dataframe
to only include rows where Area_type is "EL"
no_area = df[df["Area_type"] == "NO"].copy() #filtering the dataframe
to only include rows where Area_type is "NO"
vass_area = df[df["Area_type"] == "VASS"].copy() #filtering the
dataframe to only include rows where Area_type is "VASS"

#create 3 plots, one for each Area_type
#the plots are stacked vertically (3 rows, 1 column)
fig, axes = plt.subplots(3, 1, figsize=(12, 10),
    sharex=True, #use the same Date axis for all plots
    sharey=True #use the same Filling level scale for all plots
)

#loop through each Area_type and its corresponding plot
for ax, area_type in zip(axes, df["Area_type"].unique()):

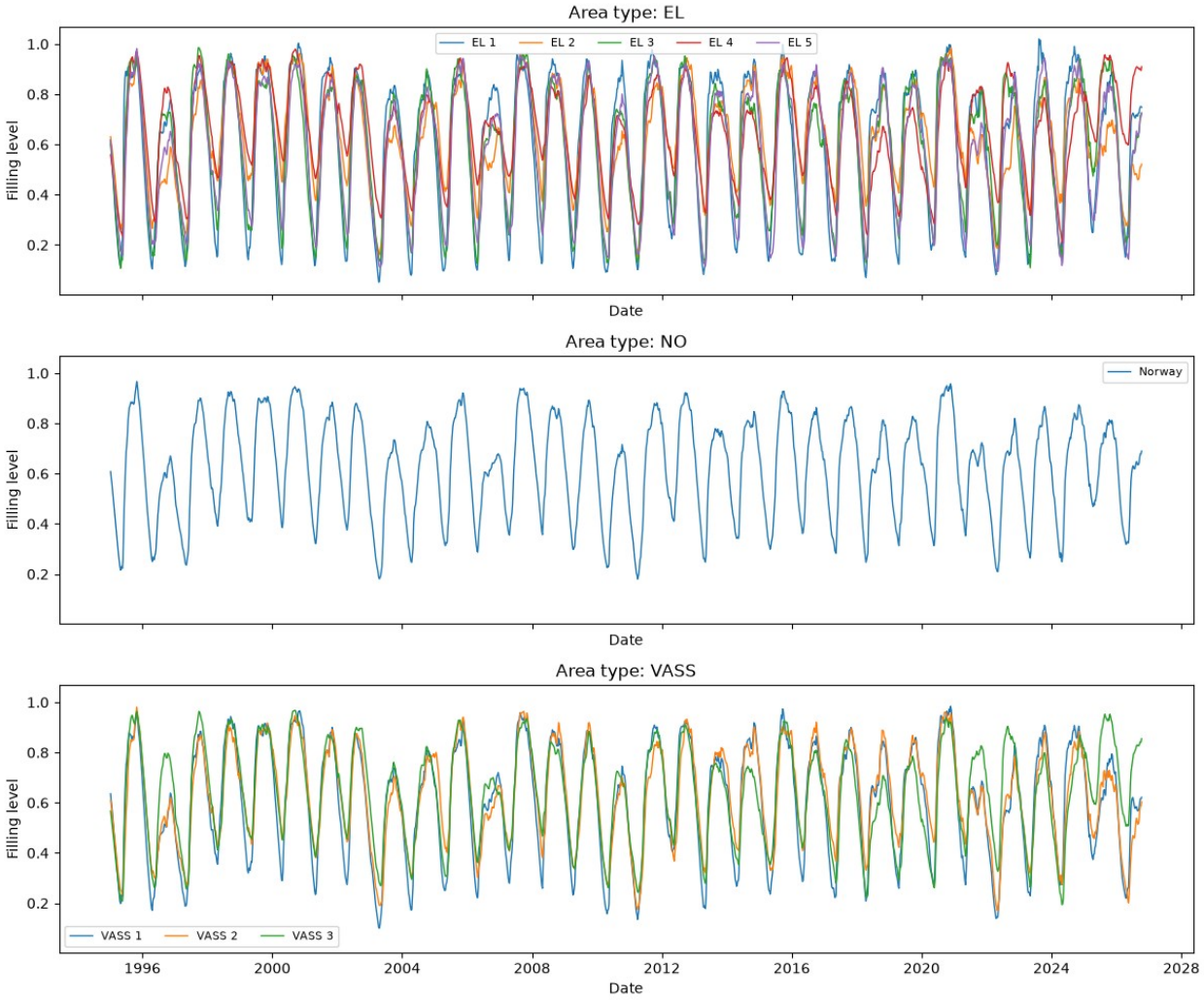
    #select only observations belonging to the current Area_type
    #and sort them chronologically by Date
    subset = df[df["Area_type"] == area_type].sort_values("Date")

    #group the data by Area_number
    for number, area in subset.groupby("Area_number"):
        #create a label for the legend
        #if the area type is NO, call it "Norway"
        #else use the Area_type and Area_number
        label = "Norway" if area_type == "NO" else f"{area_type}
{number}"

        #plot the Filling_level over Date for the current area
        ax.plot(
            area["Date"],
            area["Filling_level"],
            label=label,
            linewidth=1
        )

    #Adding a title showing which Area_type is being displayed
    ax.set_title(f"Area type: {area_type}")
    ax.set_ylabel("Filling level")
    ax.set_xlabel("Date")
    #legend showing which line belongs to which area
    ax.legend(ncol=5, fontsize=8)

plt.tight_layout()
plt.show()
```




```
#Lisy of measurement columns that we want to plot
measure_cols = ["Filling_TWh", "Filling_level_prev_week",
"Change_filling_level", "Capacity_TWh"]

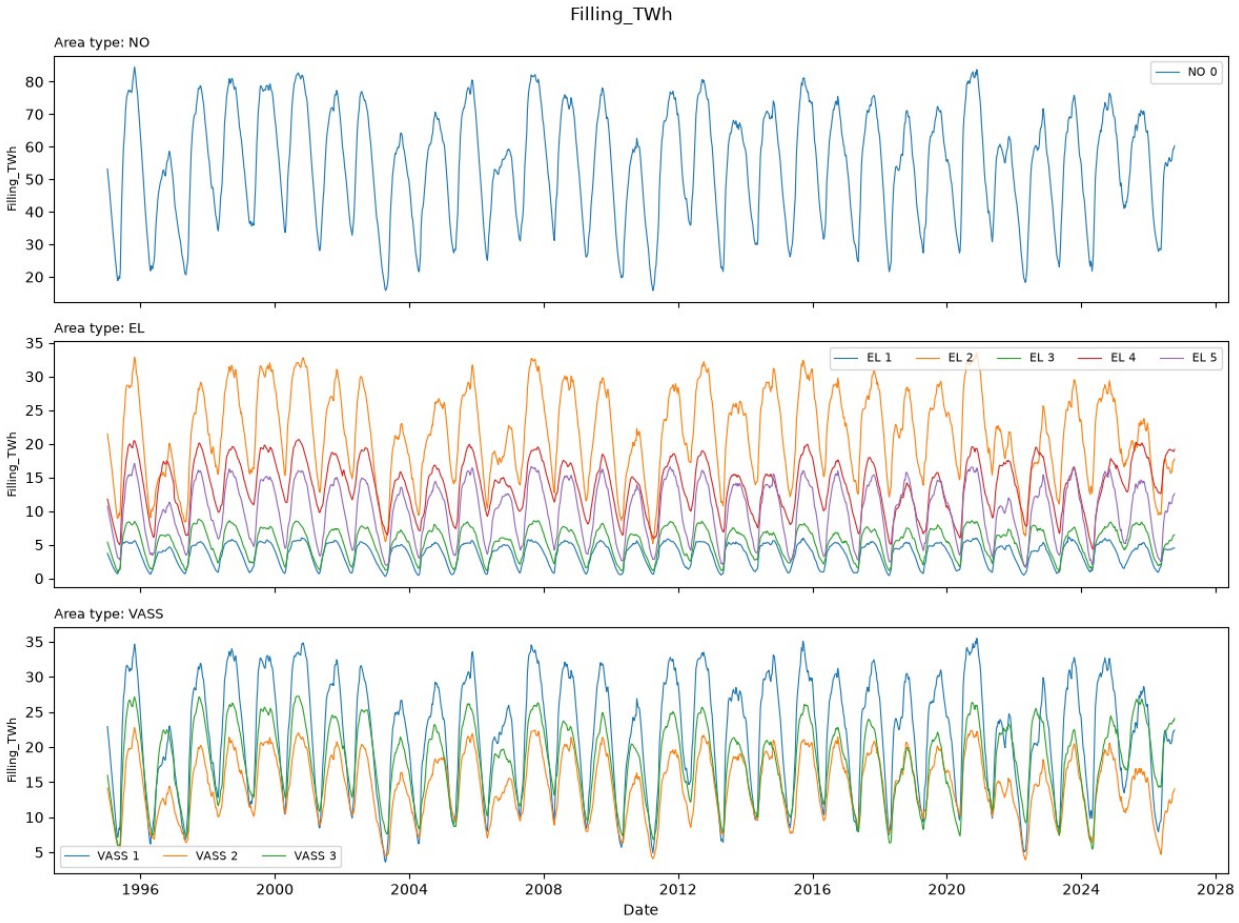
#Looping through the columns
#for each measure, a figure is created
for col in measure_cols:
    #3 subplots are created for each area type
    fig, axes = plt.subplots(3, 1, figsize=(12, 9), sharex=True)

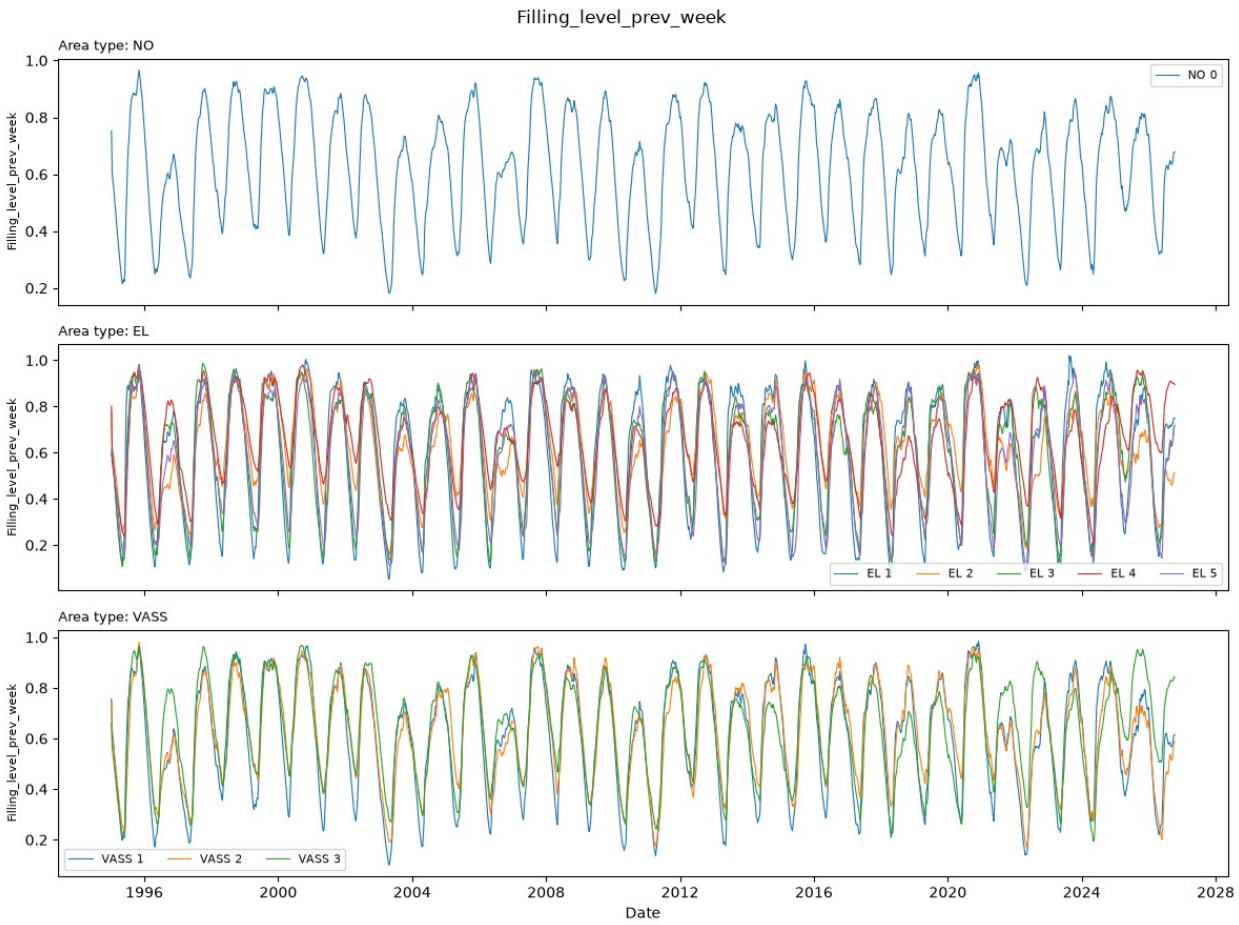
    #loop through the area tyoes and their df
    for ax, area_df, area_type in zip(axes, [no_area, el_area,
vass_area], ["NO", "EL", "VASS"]):
        #sirting the data by date and then group by Are_number
        for number, g in
area_df.sort_values("Date").groupby("Area_number"):
            #plot the selected measurement over time
            #each area number gets its line
```

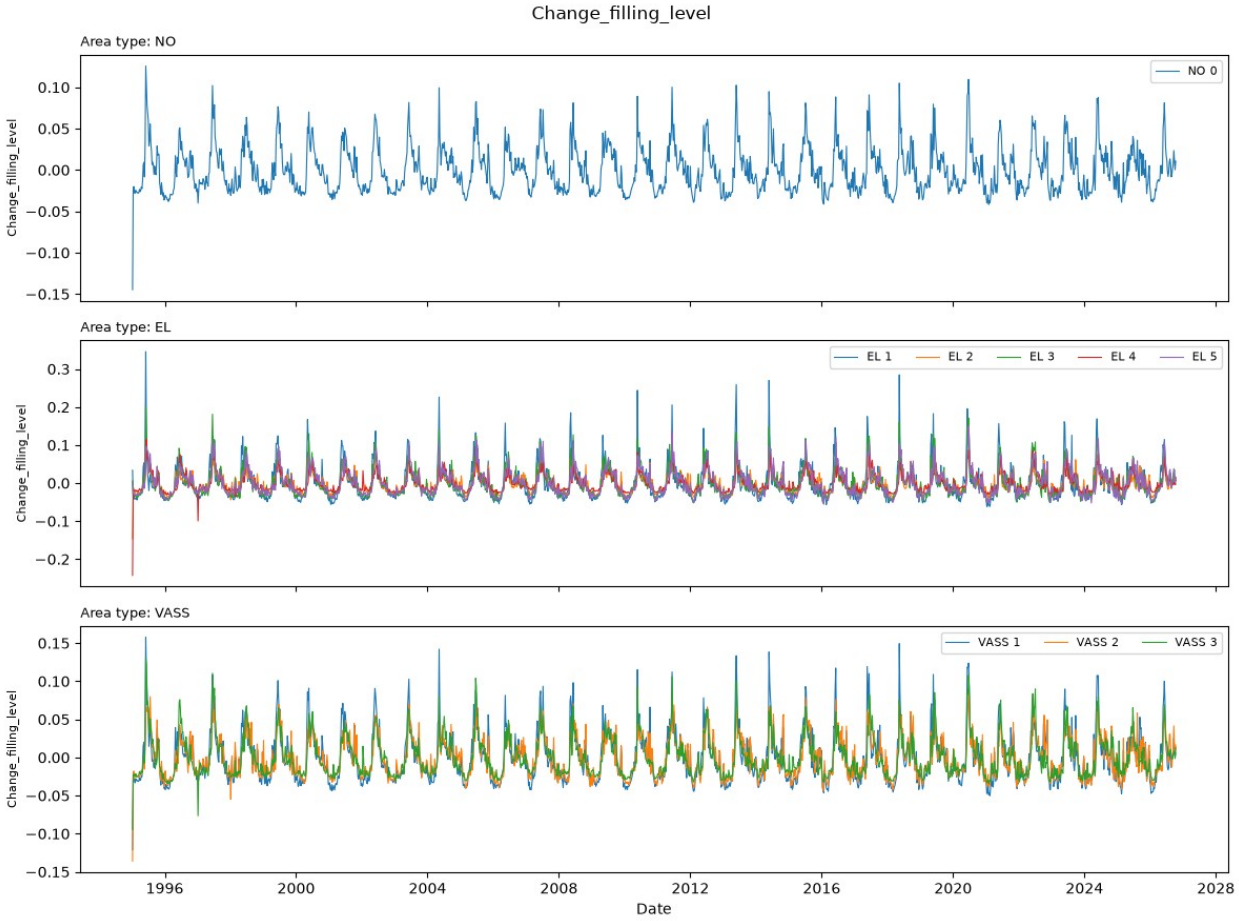
```
            ax.plot(g["Date"], g[col], label=f"{area_type} {number}",
linewidth=0.8)

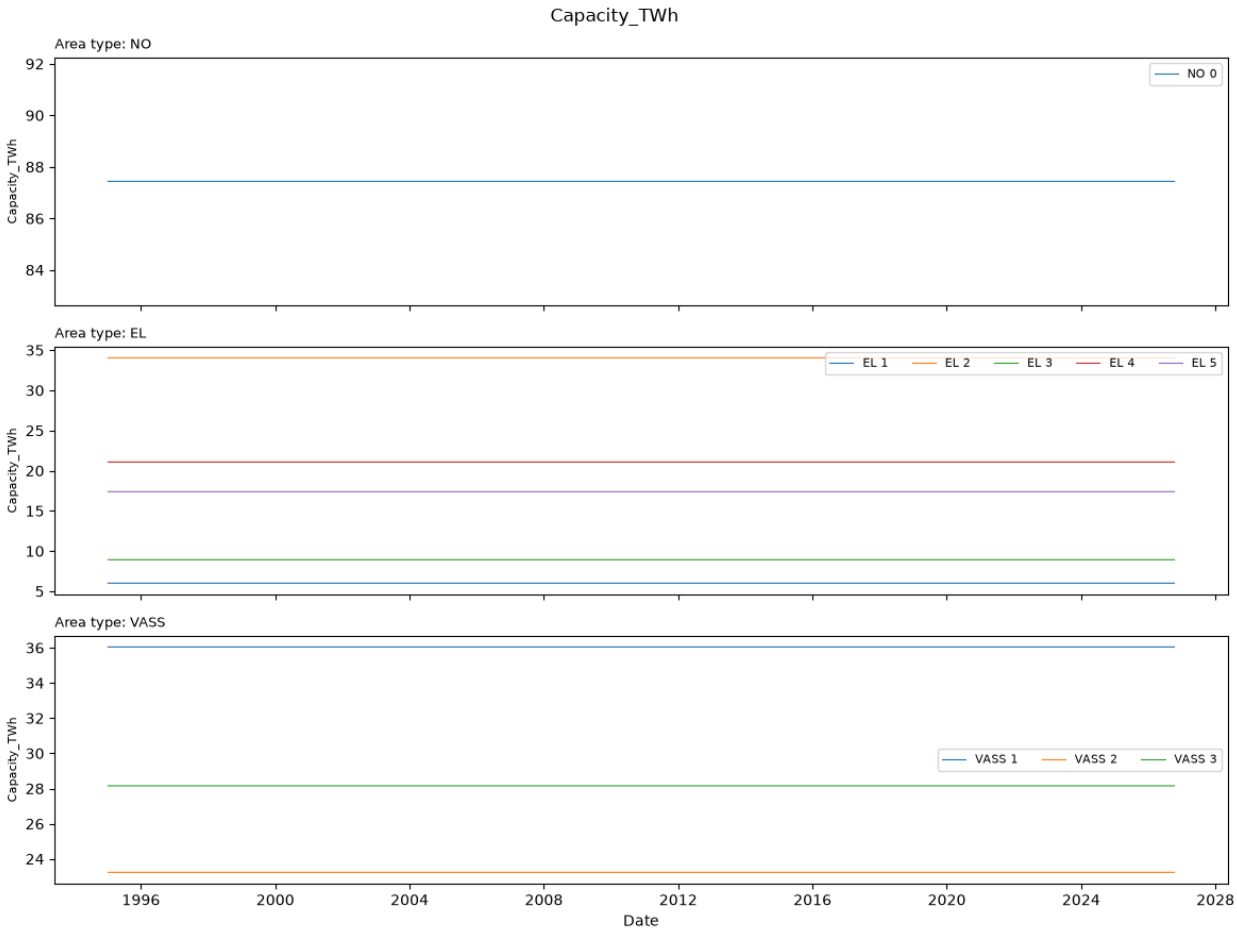
        #which Area_type is being shown
        ax.set_title(f"Area type: {area_type}", loc="left",
fontsize=9)
        ax.set_ylabel(col, fontsize=8)
        ax.legend(ncol=5, fontsize=8)

    axes[-1].set_xlabel("Date") #naming the bottom x-axis
    #naming the figure the selected measurement
    fig.suptitle(col)
    plt.tight_layout()
    plt.show()
```




Filling_level_prev_week
Area type: NO
NO 0
Area type: EL
EL 1
EL 2
EL 3
EL 4
EL 5
Area type: VASS
VASS 1
VASS 2
VASS 3
Filling_level_prev_week
1.0
0.8
0.6
0.4
0.2
1996
2000
2004
2008
2012
2016
2020
2024
2028
Date


Change_filling_level
Area type: NO
NO 0
Area type: EL
EL 1
EL 2
EL 3
EL 4
EL 5
Area type: VASS
VASS 1
VASS 2
VASS 3
Change_filling_level
Date
1996
2000
2004
2008
2012
2016
2020
2024
2028



- Filling (TWh) against date: The different colors represent the different area numbers within each area type. VASS-1, EL-2 and NO appear to follow similar trends over time, although they are on different scales. This suggests that the filling levels in these areas tend to increase and decrease in similar periods.
- Filling level in the previous week against date: Here, EL-5, VASS-2 and NO appear to follow similar trends over time. The filling level in the previous week also shows relatively consistent patterns between the different areas.
- Change in filling level against date: The changes in filling level appear to be relatively small throughout the observed period. There are some larger peaks, particularly around 1996. These peaks could indicate periods with larger changes in reservoir filling levels.
- Capacity (TWh) against date: The capacity appears to remain constant for the different area types and area numbers throughout the observed period. This is expected because the capacity represents the maximum storage capacity of the reservoirs rather than the amount of water currently stored. The graphs also show that the same area-number series are present throughout the period.

Plotting all the columns for one area_type

Only plotting all the columns for Area_type="NO" so it doesnt look as chaotic.

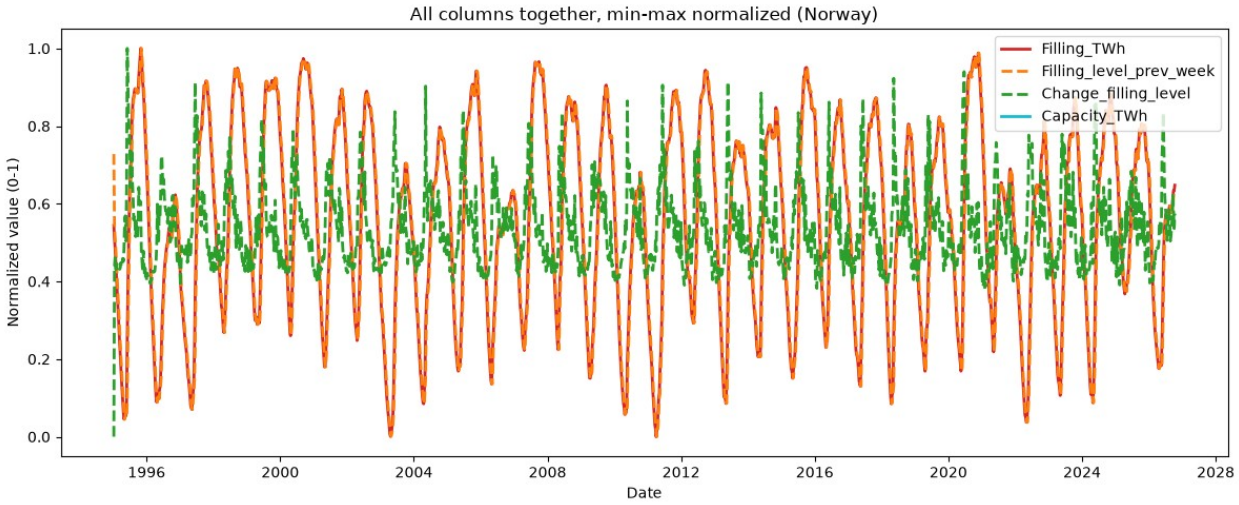
```
#normalizes all the measurement column to a scale from 0-1
#Makes the columns comparable since they have different scales.
norm = (no_area[measure_cols ] - no_area[measure_cols ].min()) /
(no_area[measure_cols ].max() - no_area[measure_cols].min())

#different line styles to easier differentiate the lines
styles = ["-", "--", "--", "-"]
#different colors for the lines
colors = ["tab:red", "tab:orange", "tab:green", "tab:cyan"]

fig, ax = plt.subplots(figsize=(12, 5))

#looping through each measurement column, linestyle and color
for col, style, color in zip(measure_cols, styles, colors):
    #Plotting the normaøozed measurements over time
    ax.plot(no_area["Date"], norm[col], label=col, linewidth=2.0,
            linestyle=style, color=color)

ax.set_xlabel("Date")
ax.set_ylabel("Normalized value (0-1)")
ax.set_title("All columns together, min-max normalized (Norway)")
ax.legend()
plt.tight_layout()
plt.show()
```




```
#calculate the correlation between the normalized measurements
correlations = norm.corr()

print(correlations)
```

```
                          Filling_TWh  Filling_level_prev_week  \
Filling_TWh                  1.000000                 0.990002
Filling_level_prev_week      0.990002                 1.000000
Change_filling_level         0.070054                -0.071353
Capacity_TWh                      NaN                      NaN

                          Change_filling_level  Capacity_TWh
Filling_TWh                           0.070054           NaN
Filling_level_prev_week              -0.071353           NaN
Change_filling_level                  1.000000           NaN
Capacity_TWh                               NaN           NaN
```

```
#describing the Capacity column for area type NO
#since we get NaN-values
print(no_area["Capacity_TWh"].describe())
```

```
count    1.658000e+03
mean     8.743758e+01
std      1.421514e-14
min      8.743758e+01
25%      8.743758e+01
50%      8.743758e+01
75%      8.743758e+01
max      8.743758e+01
Name: Capacity_TWh, dtype: float64
```

The normalized measurements show that Filling TWh and Filling level in the previous week follow very similar patterns over time. This is also supported by their very high correlation of 0.990. Change in filling level behaves differently and has very little relationship with the other two variables.

Capacity TWh is not visible in the normalized plot because its value is constant in the data for the area we examined. Since the capacity does not change, it cannot be normalized in the same way as the other variables. Therefore, we cannot use this plot to compare the development of capacity with the other measurements.

Overall, the results show that Filling TWh and Filling level in the previous week change in a very similar way, while Change in filling level follows a different pattern.

Entsoe data

```
# Reading the entose token
with open("../.streamlit/secrets.toml", "rb") as f:
    token = tomllib.load(f)["entsoe"]["token"]

client = EntsoePandasClient(api_key=token)

# Download the last two years till today
end = pd.Timestamp.today(tz="Europe/Oslo").normalize()
start = end - pd.DateOffset(years=2)
```

```
# Norwegian price areas and their neighbouring areas.
connections = [
    ("NO_1", "SE_3"),
    ("NO_2", "DK_1"),
    ("NO_2", "NL"),
    ("NO_2", "DE_LU"),
    ("NO_2", "GB"),
    ("NO_3", "SE_2"),
    ("NO_4", "SE_1"),
    ("NO_4", "SE_2"),
    ("NO_4", "FI"),
]

frames = []
failed_requests = []

for a, b in connections:
    # Query each connection in both directions.
    # Checking for-loops are running
    print("hei1")
    for src, dst in [(a, b), (b, a)]:
        # Checking for-loops are running
        print("hallo")
        try:
            flow = client.query_crossborder_flows(
                src, dst, start=start, end=end
            )

            if flow.empty:
                raise ValueError("The API returned no observations.")

            frames.append(pd.DataFrame({
                # Store timestamps consistently in UTC.
                "Time": flow.index.tz_convert("UTC"),
                "Connection": f"{a}-{b}",
                "From": src,
                "To": dst,
                "Direction": (
                    "out of Norway"
                    if src.startswith("NO_")
                    else "into Norway"
                ),
                "Flow_MW": flow.values,
            }))

        except Exception as e:
            # Record failures so incomplete downloads are clearly
identified.
            failed_requests.append((src, dst, type(e).__name__))
```

```
            print(f"Failed {src} -> {dst}: {type(e).__name__}")


# Combine and sort the downloaded data.
flows = pd.concat(frames, ignore_index=True)
flows = flows.sort_values(
    ["Connection", "Direction", "Time"]
).reset_index(drop=True)

hei1
hallo
hallo
hei1
hallo
hallo
hei1
hallo
hallo
hei1
hallo
hallo
hei1
hallo
hallo
hei1
hallo
hallo
hei1
hallo
hallo
hei1
hallo
hallo
Shape: (1038036, 6)
Connection  Direction
NO_1-SE_3   into Norway      45234
            out of Norway    45234
NO_2-DE_LU  into Norway      70080
            out of Norway    70080
NO_2-DK_1   into Norway      57762
            out of Norway    57762
NO_2-GB     into Norway      70080
            out of Norway    70080
NO_2-NL     into Norway      70080
            out of Norway    70080
NO_3-SE_2   into Norway      45234
            out of Norway    45234
```

```
NO_4-FI     into Norway      70080
            out of Norway    70080
NO_4-SE_1   into Norway      45234
            out of Norway    45234
NO_4-SE_2   into Norway      45234
            out of Norway    45234
dtype: int64
UTC timestamps: 2024-10-18 22:00:00+00:00 to 2026-10-18 21:45:00+00:00
```

```
#Save downloaded data so we dont have to rerun the above cell
#flows.to_csv("entsoe_flows.csv", index=False)

#reading the saved file of entsoe data
flows = pd.read_csv("entsoe_flows.csv")
flows["Time"] = pd.to_datetime(flows["Time"], utc=True)
```

Cassandra and pyspark connection

```
# Where java is
java_home = Path(r"C:\Program Files\Microsoft\jdk-17.0.20.101-
hotspot")

if not (java_home / "bin" / "java.exe").is_file():
    raise FileNotFoundError(f"Java installation not found:
{java_home}")


#Tell pyspark where java is
os.environ["JAVA_HOME"] = str(java_home)

os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

os.environ["HADOOP_HOME"] = r"C:\winutils-temp\hadoop-3.3.1"

# Connect to local Cassandra in docker
cluster = Cluster(["localhost"], port=9042)
session = cluster.connect()

# Create the project keyspace
#Simplestrategy with rep_fac=1 keeps one copy of each row
session.execute("""
    CREATE KEYSPACE IF NOT EXISTS ind320
    WITH replication = {
        'class': 'SimpleStrategy',
        'replication_factor': 1
    }
""")

# table for cross-border flow observations
```

```
# one row per connection, direction and timestamp
session.execute("""
    CREATE TABLE IF NOT EXISTS ind320.crossborder_flows (
        connection text,
        direction text,
        year int,
        time timestamp,
        from_area text,
        to_area text,
        flow_mw double,
        PRIMARY KEY ((connection, direction, year), time)
    )
""")

# Checking that it worked
print("Project keyspace and table are ready.")
#close the connection since we are done since we are done with this
session
cluster.shutdown()
```

```
WARNING:cassandra.cluster:Cluster.__init__ called with contact_points
specified, but no load_balancing_policy. In the next major version,
this will raise an error; please specify a load-balancing policy.
(contact_points = ['localhost'], lbp = None)
WARNING:cassandra.cluster:Downgrading core protocol version from 66 to
65 for 127.0.0.1:9042. To avoid this, it is best practice to
explicitly set Cluster(protocol_version) to the version supported by
your cluster.
https://docs.datastax.com/en/developer/python-driver/latest/api/cassan
dra/cluster.html#cassandra.cluster.Cluster.protocol_version
WARNING:cassandra.cluster:Downgrading core protocol version from 65 to
5 for 127.0.0.1:9042. To avoid this, it is best practice to explicitly
set Cluster(protocol_version) to the version supported by your
cluster.
https://docs.datastax.com/en/developer/python-driver/latest/api/cassan
dra/cluster.html#cassandra.cluster.Cluster.protocol_version

Project keyspace and table are ready.
```

```
spark = (
    SparkSession.builder
    .appName("IND320_Part2")
    .master("local[2]")
    #download the spark-cassandra connector
    .config(
        "spark.jars.packages",
        "com.datastax.spark:spark-cassandra-connector_2.12:3.5.1"
    )
    #where spark finds cassandra
    .config("spark.cassandra.connection.host", "localhost")
```

```
    .config("spark.cassandra.connection.port", "9042")
    .config("spark.sql.session.timeZone", "UTC") #sets sparks timezoen
to UTC
    .getOrCreate()
)

---------------------------------------------------------------------
-----
ConnectionResetError                      Traceback (most recent call
last)
File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\py4j\
clientserver.py:503, in ClientServerConnection.send_command(self,
command)
    502 try:
--> 503     self.socket.sendall(command.encode("utf-8"))
    504 except Exception as e:

ConnectionResetError: [WinError 10054] En eksisterende tilkobling ble
tvangslukket av den eksterne verten

During handling of the above exception, another exception occurred:

Py4JNetworkError                          Traceback (most recent call
last)
File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\py4j\
java_gateway.py:1038, in GatewayClient.send_command(self, command,
retry, binary)
   1037 try:
-> 1038     response = connection.send_command(command)
   1039     if binary:

File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\py4j\
clientserver.py:506, in ClientServerConnection.send_command(self,
command)
    505     logger.info("Error while sending or receiving.",
exc_info=True)
--> 506     raise Py4JNetworkError(
    507         "Error while sending", e, proto.ERROR_ON_SEND)
    509 try:

Py4JNetworkError: Error while sending

During handling of the above exception, another exception occurred:

ConnectionRefusedError                    Traceback (most recent call
last)
Cell In[69], line 14
      1 spark = (
      2     SparkSession.builder
      3     .appName("IND320_Part2")
```

```
      4     .master("local[2]")
      5     #download the spark-cassandra connector
      6     .config(
      7         "spark.jars.packages",
      8         "com.datastax.spark:spark-cassandra-
connector_2.12:3.5.1"
      9     )
     10     #where spark finds cassandra
     11     .config("spark.cassandra.connection.host", "localhost")
     12     .config("spark.cassandra.connection.port", "9042")
     13     .config("spark.sql.session.timeZone", "UTC") #sets sparks
timezoen to UTC
---> 14     .getOrCreate()
     15 )

File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\pyspark\
sql\session.py:503, in SparkSession.Builder.getOrCreate(self)
    500     session = SparkSession(sc, options=self._options)
    501 else:
    502     getattr(
--> 503         getattr(session._jvm, "SparkSession$"), "MODULE$"
    504     ).applyModifiableSettings(session._jsparkSession,
self._options)
    505 return session

File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\py4j\
java_gateway.py:1712, in JVMView.__getattr__(self, name)
   1709 if name == UserHelpAutoCompletion.KEY:
   1710     return UserHelpAutoCompletion()
-> 1712 answer = self._gateway_client.send_command(
   1713     proto.REFLECTION_COMMAND_NAME +
   1714     proto.REFL_GET_UNKNOWN_SUB_COMMAND_NAME + name + "\n" +
self._id +
   1715     "\n" + proto.END_COMMAND_PART)
   1716 if answer == proto.SUCCESS_PACKAGE:
   1717     return JavaPackage(name, self._gateway_client,
jvm_id=self._id)

File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\py4j\
java_gateway.py:1053, in GatewayClient.send_command(self, command,
retry, binary)
   1051 if self._should_retry(retry, connection, pne):
   1052     logging.info("Exception while sending command.",
exc_info=True)
-> 1053     response = self.send_command(command, binary=binary)
   1054 else:
   1055     logging.exception(
   1056         "Exception while sending command.")

File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\py4j\
```

```
java_gateway.py:1036, in GatewayClient.send_command(self, command,
retry, binary)
   1015 def send_command(self, command, retry=True, binary=False):
   1016     """Sends a command to the JVM. This method is not intended
to be
   1017        called directly by Py4J users. It is usually called by
   1018        :class:`JavaMember` instances.
   (...)
   1034      if `binary` is `True`.
   1035     """
-> 1036     connection = self._get_connection()
   1037     try:
   1038         response = connection.send_command(command)

File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\py4j\
clientserver.py:284, in JavaClient._get_connection(self)
    281     pass
    283 if connection is None or connection.socket is None:
--> 284     connection = self._create_new_connection()
    285 return connection

File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\py4j\
clientserver.py:291, in JavaClient._create_new_connection(self)
    287 def _create_new_connection(self):
    288     connection = ClientServerConnection(
    289         self.java_parameters, self.python_parameters,
    290         self.gateway_property, self)
--> 291     connection.connect_to_java_server()
    292     self.set_thread_connection(connection)
    293     return connection

File c:\Users\malki\anaconda3\envs\ind320\Lib\site-packages\py4j\
clientserver.py:438, in
ClientServerConnection.connect_to_java_server(self)
    435 if self.ssl_context:
    436     self.socket = self.ssl_context.wrap_socket(
    437         self.socket, server_hostname=self.java_address)
--> 438 self.socket.connect((self.java_address, self.java_port))
    439 self.stream = self.socket.makefile("rb")
    440 self.is_connected = True

ConnectionRefusedError: [WinError 10061] Kan ikke koble til fordi
målmaskinen avslo tilkobling
```

Reading data through Spark

```
#timestamps in UTC
# use Norwegian calendar years for partitioning
prepared = flows.copy()
```

```
prepared["Time"] = pd.to_datetime(prepared["Time"], utc=True)
prepared["Year"] =
prepared["Time"].dt.tz_convert("Europe/Oslo").dt.year

#defining explicitly teh spark schema
schema = StructType([
    StructField("connection", StringType(), False),
    StructField("direction", StringType(), False),
    StructField("year", IntegerType(), False),
    StructField("time", TimestampType(), False),
    StructField("from_area", StringType(), False),
    StructField("to_area", StringType(), False),
    StructField("flow_mw", DoubleType(), True),
])

# what datetype the rows shouuld be
rows = [(
        str(connection),
        str(direction),
        int(year),
        time.to_pydatetime(),
        str(from_area),
        str(to_area),
        None if pd.isna(power) else float(power),) #NaN becomes None,
else a float
    #iterate row by row
    for connection, direction, year, time, from_area, to_area, power
    in prepared[
        ["Connection", "Direction", "Year", "Time", "From", "To",
"Flow_MW"]
    ].itertuples(index=False, name=None)
]

spark.conf.set("spark.sql.session.timeZone", "UTC") #Spark interpret
timestamps as UTC
spark_flows = spark.createDataFrame(rows, schema=schema) # Build the
soark dataframe

#Append performs upserts using the primary key
#Rerunning doesnt duplicate records with the same key
(
    spark_flows.write
    .format("org.apache.spark.sql.cassandra")
    .options(keyspace="ind320", table="crossborder_flows")
    .mode("append")
    .save()
)

#read the stored entsoe data from Cassandra using Spark
stored_flows = (
```

```
    spark.read
    .format("org.apache.spark.sql.cassandra")
    .options(keyspace="ind320", table="crossborder_flows")
    .load()
)

#selecting the relevant columns
selected_flows = stored_flows.select(
    "time",
    "connection",
    "direction",
    "flow_mw"
)

#checking how many observations exist for each connection and
direction
selected_flows.groupBy(
    "connection", "direction"
).count().orderBy("connection", "direction").show(30, truncate=False)
#up to 30 rows, without truncating long strings
```

```
+----------+-------------+-----+
|connection|direction    |count|
+----------+-------------+-----+
|NO_1-SE_3 |into Norway  |45234|
|NO_1-SE_3 |out of Norway|45234|
|NO_2-DE_LU|into Norway  |70080|
|NO_2-DE_LU|out of Norway|70080|
|NO_2-DK_1 |into Norway  |57762|
|NO_2-DK_1 |out of Norway|57762|
|NO_2-GB   |into Norway  |70080|
|NO_2-GB   |out of Norway|70080|
|NO_2-NL   |into Norway  |70080|
|NO_2-NL   |out of Norway|70080|
|NO_3-SE_2 |into Norway  |45234|
|NO_3-SE_2 |out of Norway|45234|
|NO_4-FI   |into Norway  |70080|
|NO_4-FI   |out of Norway|70080|
|NO_4-SE_1 |into Norway  |45234|
|NO_4-SE_1 |out of Norway|45234|
|NO_4-SE_2 |into Norway  |45234|
|NO_4-SE_2 |out of Norway|45234|
+----------+-------------+-----+
```

We see that the same amount exported to an area is same amount imported from an area.

```
plot_data = selected_flows.toPandas() #spart dataframe to pandas
dataframe
```

```python
# Convert UTC timestamps to Norwegian local time for year/month
selection.
plot_data["time"] = (
    pd.to_datetime(plot_data["time"], utc=True)
    .dt.tz_convert("Europe/Oslo")
)

#sort by connection, direction then time
#replacing old indexing
plot_data = plot_data.sort_values(
    ["connection", "direction", "time"]
).reset_index(drop=True)

#looking at first five rows
plot_data.head()
```

	time	connection	direction	flow_mw
0	2024-10-19 00:00:00+02:00	NO_1-SE_3	into Norway	938.1250
1	2024-10-19 01:00:00+02:00	NO_1-SE_3	into Norway	760.9550
2	2024-10-19 02:00:00+02:00	NO_1-SE_3	into Norway	723.5400
3	2024-10-19 03:00:00+02:00	NO_1-SE_3	into Norway	742.2175
4	2024-10-19 04:00:00+02:00	NO_1-SE_3	into Norway	711.7750

```python
#selecting the Norway–Sweden connection
#area 1 for norway and 3 for Sweden
data = plot_data[
    plot_data["connection"] == "NO_1-SE_3"
].copy()

#one row per direction per timepoint
keys = ["direction", "time"]
# removing duplicates
data = data.drop_duplicates(keys).sort_values(keys)

#estimate each interval from the next timestamp in the same direction
next_time = data.groupby("direction")["time"].shift(-1)
#the interval length in hours
data["hours"] = (next_time - data["time"]).dt.total_seconds() / 3600

#only keeping 2025 data
data = data[data["time"].dt.year == 2025].copy()

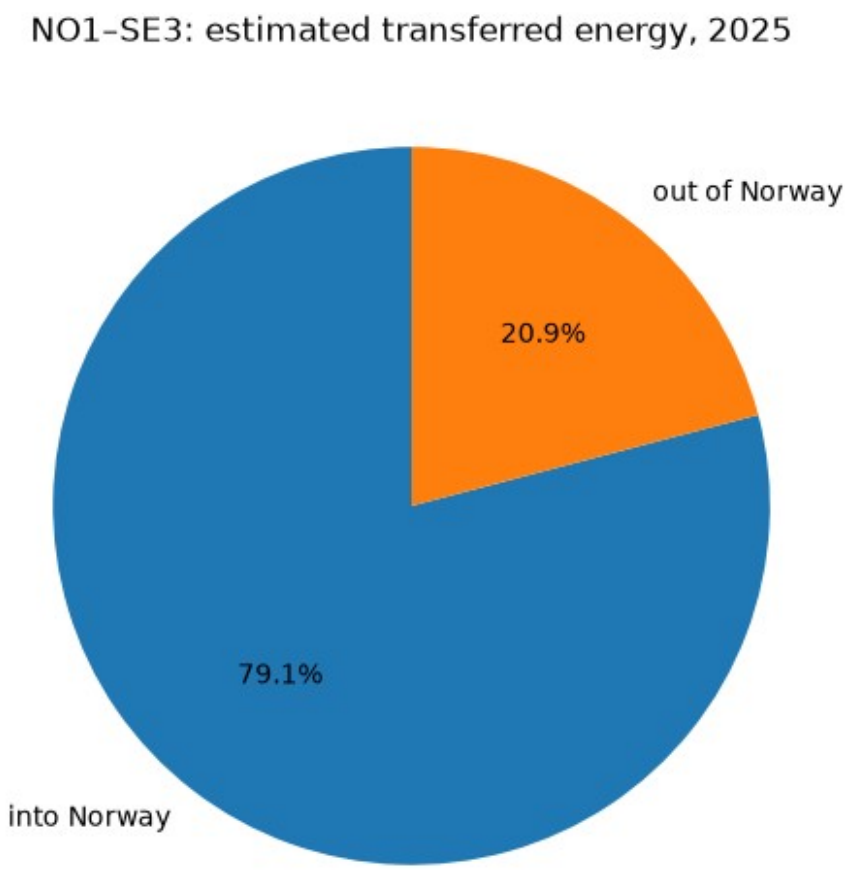
#looking at 15-minutes and 1-hour intervals and must have a flow-value
valid = data["hours"].isin([0.25, 1.0]) & data["flow_mw"].notna()
#printing out the rows not fulfilling above req.
print("Rows excluded because of missing values or unusual gaps:",
(~valid).sum())
#dropping the rows
data = data[valid].copy()
```

```python
#converting power to energy
#then total imports and exports
#energy = power * time
data["energy_mwh"] = data["flow_mw"] * data["hours"]
#total energy per direction
annual = data.groupby("direction")["energy_mwh"].sum()

print("Estimated energy (MWh):")
print(annual)

#plot of the direction shares for NO1-SE3
annual.plot.pie(autopct="%1.1f%%", startangle=90, figsize=(6, 6))
plt.title("NO1–SE3: estimated transferred energy, 2025")
plt.show()
```

```
Rows excluded because of missing values or unusual gaps: 2
Estimated energy (MWh):
direction
into Norway      6.138456e+06
out of Norway    1.623833e+06
Name: energy_mwh, dtype: float64
```



Most energy to Norway area 1, is imported from Sweden area 3 with almost 80% energy.

```
#select january for the NO1-SE3 connection
november = plot_data[
    (plot_data["connection"] == "NO_1-SE_3") &
    (plot_data["time"].dt.year == 2025) &
    (plot_data["time"].dt.month == 11)
].copy()

#only looking at unique rows and dropping duplicates
november = november.drop_duplicates(
    ["time", "direction", "flow_mw"]
).sort_values("time")

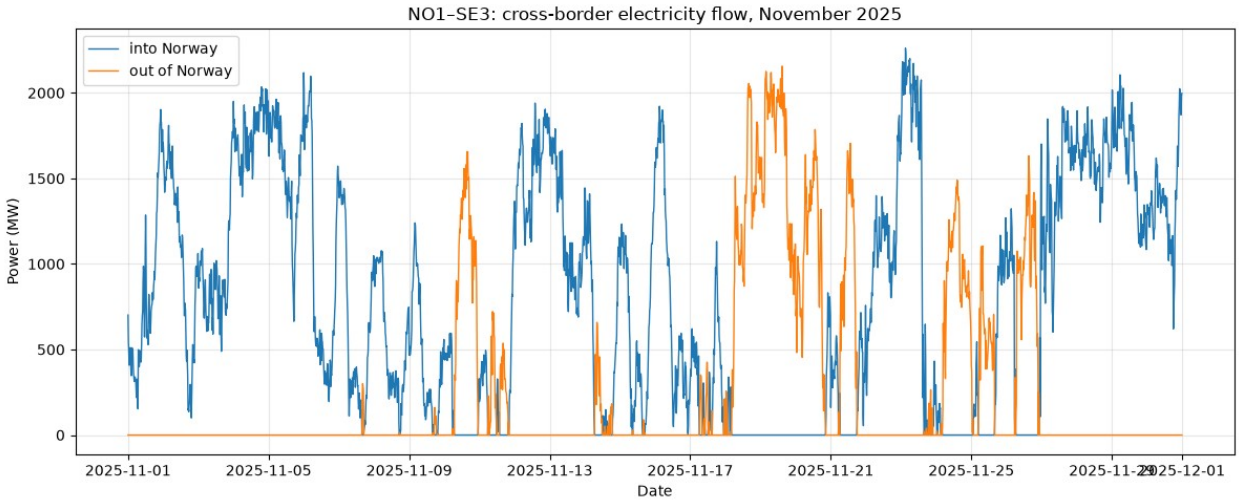
fig, ax = plt.subplots(figsize=(12, 5))

#plotting the data by import/export
for direction, group in november.groupby("direction"):
    ax.plot(
        group["time"],
        group["flow_mw"],
```

```
        label=direction,
        linewidth=1,
    )

ax.set_title("NO1–SE3: cross-border electricity flow, November 2025")
ax.set_xlabel("Date")
ax.set_ylabel("Power (MW)")
ax.legend()
ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()
```




```
WARNING:cassandra.cluster:Host ::1:9042 has been marked down
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 1.86 seconds: Connection to ::1:9042 was closed
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 3.76 seconds: Connection to ::1:9042 was closed
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 7.44 seconds: Connection to ::1:9042 was closed
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 14.24 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 31.04 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 71.68 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
```

```
scheduling retry in 119.04 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 253.44 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 558.08 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 552.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 528.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 522.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 540.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 540.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 582.0 seconds: [Errno 10061] Tried connecting to
```

```
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 510.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 558.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 546.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 546.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 552.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 516.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 510.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
```

```
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 588.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 570.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 540.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 522.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 540.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 564.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
```

```
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 540.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 528.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 546.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 516.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 516.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
```

```
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 510.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
WARNING:cassandra.pool:Error attempting to reconnect to ::1:9042,
scheduling retry in 600.0 seconds: [Errno 10061] Tried connecting to
[('::1', 9042, 0, 0)]. Last error: Kan ikke koble til fordi
målmaskinen avslo tilkobling
```

Spark into Mongodb

```python
#opening the secrets file
with open("../.streamlit/secrets.toml", "rb") as f:
    secrets = tomllib.load(f)

#connecting using uri
mongo_client = MongoClient(
    secrets["mongo"]["uri"],
    tz_aware=True, # For timestamps
)

#checking it connected
mongo_client.admin.command("ping")
print("Connected to MongoDB!")

#choosing the project database and collection
collection = mongo_client["ind320"]["crossborder_flows"]
```

```
Connected to MongoDB!
```

```python
#use all data extracted from Cassandra through Spark
mongo_data = plot_data[
    ["time", "connection", "direction", "flow_mw"]
].copy()
```

```python
#interate in bactches of 1000 rows
for start in range(0, len(mongo_data), 1000):
    batch = mongo_data.iloc[start:start + 1000]
    operations = []
    #iterating through each bath
    for row in batch.itertuples(index=False):
        #building the mongo document
        record = {
            "time": row.time.to_pydatetime(),  #need it in python
format
            "connection": row.connection,
            "direction": row.direction,
            "flow_mw": None if pd.isna(row.flow_mw) else
float(row.flow_mw), #NaN becomes None or float
        }

        key = {column: record[column] for column in keys} #our keys

        #upsert updates if the key exists
        #or else inserts it
        operations.append(
            UpdateOne(key, {"$set": record}, upsert=True)
        )

    #sending batch as one bulk write
    collection.bulk_write(operations, ordered=False)

    #To see progress
    print(
        f"Processed {min(start + 1000, len(mongo_data)):,} "
        f"of {len(mongo_data):,} rows",
        flush=True,
    )

print(f"Upload finished: {len(mongo_data):,} records processed.")
```

```
Processed 1,000 of 1,038,036 rows
Processed 2,000 of 1,038,036 rows
Processed 3,000 of 1,038,036 rows
Processed 4,000 of 1,038,036 rows
Processed 5,000 of 1,038,036 rows
Processed 6,000 of 1,038,036 rows
Processed 7,000 of 1,038,036 rows
Processed 8,000 of 1,038,036 rows
Processed 9,000 of 1,038,036 rows
Processed 10,000 of 1,038,036 rows
Processed 11,000 of 1,038,036 rows
Processed 12,000 of 1,038,036 rows
Processed 13,000 of 1,038,036 rows
Processed 14,000 of 1,038,036 rows
```

```
Processed 15,000 of 1,038,036 rows
Processed 16,000 of 1,038,036 rows
Processed 17,000 of 1,038,036 rows
Processed 18,000 of 1,038,036 rows
Processed 19,000 of 1,038,036 rows
Processed 20,000 of 1,038,036 rows
Processed 21,000 of 1,038,036 rows
Processed 22,000 of 1,038,036 rows
Processed 23,000 of 1,038,036 rows
Processed 24,000 of 1,038,036 rows
Processed 25,000 of 1,038,036 rows
Processed 26,000 of 1,038,036 rows
Processed 27,000 of 1,038,036 rows
Processed 28,000 of 1,038,036 rows
Processed 29,000 of 1,038,036 rows
Processed 30,000 of 1,038,036 rows
Processed 31,000 of 1,038,036 rows
Processed 32,000 of 1,038,036 rows
Processed 33,000 of 1,038,036 rows
Processed 34,000 of 1,038,036 rows
Processed 35,000 of 1,038,036 rows
Processed 36,000 of 1,038,036 rows
Processed 37,000 of 1,038,036 rows
Processed 38,000 of 1,038,036 rows
Processed 39,000 of 1,038,036 rows
Processed 40,000 of 1,038,036 rows
Processed 41,000 of 1,038,036 rows
Processed 42,000 of 1,038,036 rows
Processed 43,000 of 1,038,036 rows
Processed 44,000 of 1,038,036 rows
Processed 45,000 of 1,038,036 rows
Processed 46,000 of 1,038,036 rows
Processed 47,000 of 1,038,036 rows
Processed 48,000 of 1,038,036 rows
Processed 49,000 of 1,038,036 rows
Processed 50,000 of 1,038,036 rows
Processed 51,000 of 1,038,036 rows
Processed 52,000 of 1,038,036 rows
Processed 53,000 of 1,038,036 rows
Processed 54,000 of 1,038,036 rows
Processed 55,000 of 1,038,036 rows
Processed 56,000 of 1,038,036 rows
Processed 57,000 of 1,038,036 rows
Processed 58,000 of 1,038,036 rows
Processed 59,000 of 1,038,036 rows
Processed 60,000 of 1,038,036 rows
Processed 61,000 of 1,038,036 rows
Processed 62,000 of 1,038,036 rows
Processed 63,000 of 1,038,036 rows
```

```
Processed 64,000 of 1,038,036 rows
Processed 65,000 of 1,038,036 rows
Processed 66,000 of 1,038,036 rows
Processed 67,000 of 1,038,036 rows
Processed 68,000 of 1,038,036 rows
Processed 69,000 of 1,038,036 rows
Processed 70,000 of 1,038,036 rows
Processed 71,000 of 1,038,036 rows
Processed 72,000 of 1,038,036 rows
Processed 73,000 of 1,038,036 rows
Processed 74,000 of 1,038,036 rows
Processed 75,000 of 1,038,036 rows
Processed 76,000 of 1,038,036 rows
Processed 77,000 of 1,038,036 rows
Processed 78,000 of 1,038,036 rows
Processed 79,000 of 1,038,036 rows
Processed 80,000 of 1,038,036 rows
Processed 81,000 of 1,038,036 rows
Processed 82,000 of 1,038,036 rows
Processed 83,000 of 1,038,036 rows
Processed 84,000 of 1,038,036 rows
Processed 85,000 of 1,038,036 rows
Processed 86,000 of 1,038,036 rows
Processed 87,000 of 1,038,036 rows
Processed 88,000 of 1,038,036 rows
Processed 89,000 of 1,038,036 rows
Processed 90,000 of 1,038,036 rows
Processed 91,000 of 1,038,036 rows
Processed 92,000 of 1,038,036 rows
Processed 93,000 of 1,038,036 rows
Processed 94,000 of 1,038,036 rows
Processed 95,000 of 1,038,036 rows
Processed 96,000 of 1,038,036 rows
Processed 97,000 of 1,038,036 rows
Processed 98,000 of 1,038,036 rows
Processed 99,000 of 1,038,036 rows
Processed 100,000 of 1,038,036 rows
Processed 101,000 of 1,038,036 rows
Processed 102,000 of 1,038,036 rows
Processed 103,000 of 1,038,036 rows
Processed 104,000 of 1,038,036 rows
Processed 105,000 of 1,038,036 rows
Processed 106,000 of 1,038,036 rows
Processed 107,000 of 1,038,036 rows
Processed 108,000 of 1,038,036 rows
Processed 109,000 of 1,038,036 rows
Processed 110,000 of 1,038,036 rows
Processed 111,000 of 1,038,036 rows
Processed 112,000 of 1,038,036 rows
```

```
Processed 113,000 of 1,038,036 rows
Processed 114,000 of 1,038,036 rows
Processed 115,000 of 1,038,036 rows
Processed 116,000 of 1,038,036 rows
Processed 117,000 of 1,038,036 rows
Processed 118,000 of 1,038,036 rows
Processed 119,000 of 1,038,036 rows
Processed 120,000 of 1,038,036 rows
Processed 121,000 of 1,038,036 rows
Processed 122,000 of 1,038,036 rows
Processed 123,000 of 1,038,036 rows
Processed 124,000 of 1,038,036 rows
Processed 125,000 of 1,038,036 rows
Processed 126,000 of 1,038,036 rows
Processed 127,000 of 1,038,036 rows
Processed 128,000 of 1,038,036 rows
Processed 129,000 of 1,038,036 rows
Processed 130,000 of 1,038,036 rows
Processed 131,000 of 1,038,036 rows
Processed 132,000 of 1,038,036 rows
Processed 133,000 of 1,038,036 rows
Processed 134,000 of 1,038,036 rows
Processed 135,000 of 1,038,036 rows
Processed 136,000 of 1,038,036 rows
Processed 137,000 of 1,038,036 rows
Processed 138,000 of 1,038,036 rows
Processed 139,000 of 1,038,036 rows
Processed 140,000 of 1,038,036 rows
Processed 141,000 of 1,038,036 rows
Processed 142,000 of 1,038,036 rows
Processed 143,000 of 1,038,036 rows
Processed 144,000 of 1,038,036 rows
Processed 145,000 of 1,038,036 rows
Processed 146,000 of 1,038,036 rows
Processed 147,000 of 1,038,036 rows
Processed 148,000 of 1,038,036 rows
Processed 149,000 of 1,038,036 rows
Processed 150,000 of 1,038,036 rows
Processed 151,000 of 1,038,036 rows
Processed 152,000 of 1,038,036 rows
Processed 153,000 of 1,038,036 rows
Processed 154,000 of 1,038,036 rows
Processed 155,000 of 1,038,036 rows
Processed 156,000 of 1,038,036 rows
Processed 157,000 of 1,038,036 rows
Processed 158,000 of 1,038,036 rows
Processed 159,000 of 1,038,036 rows
Processed 160,000 of 1,038,036 rows
Processed 161,000 of 1,038,036 rows
```

```
Processed 162,000 of 1,038,036 rows
Processed 163,000 of 1,038,036 rows
Processed 164,000 of 1,038,036 rows
Processed 165,000 of 1,038,036 rows
Processed 166,000 of 1,038,036 rows
Processed 167,000 of 1,038,036 rows
Processed 168,000 of 1,038,036 rows
Processed 169,000 of 1,038,036 rows
Processed 170,000 of 1,038,036 rows
Processed 171,000 of 1,038,036 rows
Processed 172,000 of 1,038,036 rows
Processed 173,000 of 1,038,036 rows
Processed 174,000 of 1,038,036 rows
Processed 175,000 of 1,038,036 rows
Processed 176,000 of 1,038,036 rows
Processed 177,000 of 1,038,036 rows
Processed 178,000 of 1,038,036 rows
Processed 179,000 of 1,038,036 rows
Processed 180,000 of 1,038,036 rows
Processed 181,000 of 1,038,036 rows
Processed 182,000 of 1,038,036 rows
Processed 183,000 of 1,038,036 rows
Processed 184,000 of 1,038,036 rows
Processed 185,000 of 1,038,036 rows
Processed 186,000 of 1,038,036 rows
Processed 187,000 of 1,038,036 rows
Processed 188,000 of 1,038,036 rows
Processed 189,000 of 1,038,036 rows
Processed 190,000 of 1,038,036 rows
Processed 191,000 of 1,038,036 rows
Processed 192,000 of 1,038,036 rows
Processed 193,000 of 1,038,036 rows
Processed 194,000 of 1,038,036 rows
Processed 195,000 of 1,038,036 rows
Processed 196,000 of 1,038,036 rows
Processed 197,000 of 1,038,036 rows
Processed 198,000 of 1,038,036 rows
Processed 199,000 of 1,038,036 rows
Processed 200,000 of 1,038,036 rows
Processed 201,000 of 1,038,036 rows
Processed 202,000 of 1,038,036 rows
Processed 203,000 of 1,038,036 rows
Processed 204,000 of 1,038,036 rows
Processed 205,000 of 1,038,036 rows
Processed 206,000 of 1,038,036 rows
Processed 207,000 of 1,038,036 rows
Processed 208,000 of 1,038,036 rows
Processed 209,000 of 1,038,036 rows
Processed 210,000 of 1,038,036 rows
```

```
Processed 211,000 of 1,038,036 rows
Processed 212,000 of 1,038,036 rows
Processed 213,000 of 1,038,036 rows
Processed 214,000 of 1,038,036 rows
Processed 215,000 of 1,038,036 rows
Processed 216,000 of 1,038,036 rows
Processed 217,000 of 1,038,036 rows
Processed 218,000 of 1,038,036 rows
Processed 219,000 of 1,038,036 rows
Processed 220,000 of 1,038,036 rows
Processed 221,000 of 1,038,036 rows
Processed 222,000 of 1,038,036 rows
Processed 223,000 of 1,038,036 rows
Processed 224,000 of 1,038,036 rows
Processed 225,000 of 1,038,036 rows
Processed 226,000 of 1,038,036 rows
Processed 227,000 of 1,038,036 rows
Processed 228,000 of 1,038,036 rows
Processed 229,000 of 1,038,036 rows
Processed 230,000 of 1,038,036 rows
Processed 231,000 of 1,038,036 rows
Processed 232,000 of 1,038,036 rows
Processed 233,000 of 1,038,036 rows
Processed 234,000 of 1,038,036 rows
Processed 235,000 of 1,038,036 rows
Processed 236,000 of 1,038,036 rows
Processed 237,000 of 1,038,036 rows
Processed 238,000 of 1,038,036 rows
Processed 239,000 of 1,038,036 rows
Processed 240,000 of 1,038,036 rows
Processed 241,000 of 1,038,036 rows
Processed 242,000 of 1,038,036 rows
Processed 243,000 of 1,038,036 rows
Processed 244,000 of 1,038,036 rows
Processed 245,000 of 1,038,036 rows
Processed 246,000 of 1,038,036 rows
Processed 247,000 of 1,038,036 rows
Processed 248,000 of 1,038,036 rows
Processed 249,000 of 1,038,036 rows
Processed 250,000 of 1,038,036 rows
Processed 251,000 of 1,038,036 rows
Processed 252,000 of 1,038,036 rows
Processed 253,000 of 1,038,036 rows
Processed 254,000 of 1,038,036 rows
Processed 255,000 of 1,038,036 rows
Processed 256,000 of 1,038,036 rows
Processed 257,000 of 1,038,036 rows
Processed 258,000 of 1,038,036 rows
Processed 259,000 of 1,038,036 rows
```

```
Processed 260,000 of 1,038,036 rows
Processed 261,000 of 1,038,036 rows
Processed 262,000 of 1,038,036 rows
Processed 263,000 of 1,038,036 rows
Processed 264,000 of 1,038,036 rows
Processed 265,000 of 1,038,036 rows
Processed 266,000 of 1,038,036 rows
Processed 267,000 of 1,038,036 rows
Processed 268,000 of 1,038,036 rows
Processed 269,000 of 1,038,036 rows
Processed 270,000 of 1,038,036 rows
Processed 271,000 of 1,038,036 rows
Processed 272,000 of 1,038,036 rows
Processed 273,000 of 1,038,036 rows
Processed 274,000 of 1,038,036 rows
Processed 275,000 of 1,038,036 rows
Processed 276,000 of 1,038,036 rows
Processed 277,000 of 1,038,036 rows
Processed 278,000 of 1,038,036 rows
Processed 279,000 of 1,038,036 rows
Processed 280,000 of 1,038,036 rows
Processed 281,000 of 1,038,036 rows
Processed 282,000 of 1,038,036 rows
Processed 283,000 of 1,038,036 rows
Processed 284,000 of 1,038,036 rows
Processed 285,000 of 1,038,036 rows
Processed 286,000 of 1,038,036 rows
Processed 287,000 of 1,038,036 rows
Processed 288,000 of 1,038,036 rows
Processed 289,000 of 1,038,036 rows
Processed 290,000 of 1,038,036 rows
Processed 291,000 of 1,038,036 rows
Processed 292,000 of 1,038,036 rows
Processed 293,000 of 1,038,036 rows
Processed 294,000 of 1,038,036 rows
Processed 295,000 of 1,038,036 rows
Processed 296,000 of 1,038,036 rows
Processed 297,000 of 1,038,036 rows
Processed 298,000 of 1,038,036 rows
Processed 299,000 of 1,038,036 rows
Processed 300,000 of 1,038,036 rows
Processed 301,000 of 1,038,036 rows
Processed 302,000 of 1,038,036 rows
Processed 303,000 of 1,038,036 rows
Processed 304,000 of 1,038,036 rows
Processed 305,000 of 1,038,036 rows
Processed 306,000 of 1,038,036 rows
Processed 307,000 of 1,038,036 rows
Processed 308,000 of 1,038,036 rows
```

```
Processed 309,000 of 1,038,036 rows
Processed 310,000 of 1,038,036 rows
Processed 311,000 of 1,038,036 rows
Processed 312,000 of 1,038,036 rows
Processed 313,000 of 1,038,036 rows
Processed 314,000 of 1,038,036 rows
Processed 315,000 of 1,038,036 rows
Processed 316,000 of 1,038,036 rows
Processed 317,000 of 1,038,036 rows
Processed 318,000 of 1,038,036 rows
Processed 319,000 of 1,038,036 rows
Processed 320,000 of 1,038,036 rows
Processed 321,000 of 1,038,036 rows
Processed 322,000 of 1,038,036 rows
Processed 323,000 of 1,038,036 rows
Processed 324,000 of 1,038,036 rows
Processed 325,000 of 1,038,036 rows
Processed 326,000 of 1,038,036 rows
Processed 327,000 of 1,038,036 rows
Processed 328,000 of 1,038,036 rows
Processed 329,000 of 1,038,036 rows
Processed 330,000 of 1,038,036 rows
Processed 331,000 of 1,038,036 rows
Processed 332,000 of 1,038,036 rows
Processed 333,000 of 1,038,036 rows
Processed 334,000 of 1,038,036 rows
Processed 335,000 of 1,038,036 rows
Processed 336,000 of 1,038,036 rows
Processed 337,000 of 1,038,036 rows
Processed 338,000 of 1,038,036 rows
Processed 339,000 of 1,038,036 rows
Processed 340,000 of 1,038,036 rows
Processed 341,000 of 1,038,036 rows
Processed 342,000 of 1,038,036 rows
Processed 343,000 of 1,038,036 rows
Processed 344,000 of 1,038,036 rows
Processed 345,000 of 1,038,036 rows
Processed 346,000 of 1,038,036 rows
Processed 347,000 of 1,038,036 rows
Processed 348,000 of 1,038,036 rows
Processed 349,000 of 1,038,036 rows
Processed 350,000 of 1,038,036 rows
Processed 351,000 of 1,038,036 rows
Processed 352,000 of 1,038,036 rows
Processed 353,000 of 1,038,036 rows
Processed 354,000 of 1,038,036 rows
Processed 355,000 of 1,038,036 rows
Processed 356,000 of 1,038,036 rows
Processed 357,000 of 1,038,036 rows
```

```
Processed 358,000 of 1,038,036 rows
Processed 359,000 of 1,038,036 rows
Processed 360,000 of 1,038,036 rows
Processed 361,000 of 1,038,036 rows
Processed 362,000 of 1,038,036 rows
Processed 363,000 of 1,038,036 rows
Processed 364,000 of 1,038,036 rows
Processed 365,000 of 1,038,036 rows
Processed 366,000 of 1,038,036 rows
Processed 367,000 of 1,038,036 rows
Processed 368,000 of 1,038,036 rows
Processed 369,000 of 1,038,036 rows
Processed 370,000 of 1,038,036 rows
Processed 371,000 of 1,038,036 rows
Processed 372,000 of 1,038,036 rows
Processed 373,000 of 1,038,036 rows
Processed 374,000 of 1,038,036 rows
Processed 375,000 of 1,038,036 rows
Processed 376,000 of 1,038,036 rows
Processed 377,000 of 1,038,036 rows
Processed 378,000 of 1,038,036 rows
Processed 379,000 of 1,038,036 rows
Processed 380,000 of 1,038,036 rows
Processed 381,000 of 1,038,036 rows
Processed 382,000 of 1,038,036 rows
Processed 383,000 of 1,038,036 rows
Processed 384,000 of 1,038,036 rows
Processed 385,000 of 1,038,036 rows
Processed 386,000 of 1,038,036 rows
Processed 387,000 of 1,038,036 rows
Processed 388,000 of 1,038,036 rows
Processed 389,000 of 1,038,036 rows
Processed 390,000 of 1,038,036 rows
Processed 391,000 of 1,038,036 rows
Processed 392,000 of 1,038,036 rows
Processed 393,000 of 1,038,036 rows
Processed 394,000 of 1,038,036 rows
Processed 395,000 of 1,038,036 rows
Processed 396,000 of 1,038,036 rows
Processed 397,000 of 1,038,036 rows
Processed 398,000 of 1,038,036 rows
Processed 399,000 of 1,038,036 rows
Processed 400,000 of 1,038,036 rows
Processed 401,000 of 1,038,036 rows
Processed 402,000 of 1,038,036 rows
Processed 403,000 of 1,038,036 rows
Processed 404,000 of 1,038,036 rows
Processed 405,000 of 1,038,036 rows
Processed 406,000 of 1,038,036 rows
```

```
Processed 407,000 of 1,038,036 rows
Processed 408,000 of 1,038,036 rows
Processed 409,000 of 1,038,036 rows
Processed 410,000 of 1,038,036 rows
Processed 411,000 of 1,038,036 rows
Processed 412,000 of 1,038,036 rows
Processed 413,000 of 1,038,036 rows
Processed 414,000 of 1,038,036 rows
Processed 415,000 of 1,038,036 rows
Processed 416,000 of 1,038,036 rows
Processed 417,000 of 1,038,036 rows
Processed 418,000 of 1,038,036 rows
Processed 419,000 of 1,038,036 rows
Processed 420,000 of 1,038,036 rows
Processed 421,000 of 1,038,036 rows
Processed 422,000 of 1,038,036 rows
Processed 423,000 of 1,038,036 rows
Processed 424,000 of 1,038,036 rows
Processed 425,000 of 1,038,036 rows
Processed 426,000 of 1,038,036 rows
Processed 427,000 of 1,038,036 rows
Processed 428,000 of 1,038,036 rows
Processed 429,000 of 1,038,036 rows
Processed 430,000 of 1,038,036 rows
Processed 431,000 of 1,038,036 rows
Processed 432,000 of 1,038,036 rows
Processed 433,000 of 1,038,036 rows
Processed 434,000 of 1,038,036 rows
Processed 435,000 of 1,038,036 rows
Processed 436,000 of 1,038,036 rows
Processed 437,000 of 1,038,036 rows
Processed 438,000 of 1,038,036 rows
Processed 439,000 of 1,038,036 rows
Processed 440,000 of 1,038,036 rows
Processed 441,000 of 1,038,036 rows
Processed 442,000 of 1,038,036 rows
Processed 443,000 of 1,038,036 rows
Processed 444,000 of 1,038,036 rows
Processed 445,000 of 1,038,036 rows
Processed 446,000 of 1,038,036 rows
Processed 447,000 of 1,038,036 rows
Processed 448,000 of 1,038,036 rows
Processed 449,000 of 1,038,036 rows
Processed 450,000 of 1,038,036 rows
Processed 451,000 of 1,038,036 rows
Processed 452,000 of 1,038,036 rows
Processed 453,000 of 1,038,036 rows
Processed 454,000 of 1,038,036 rows
Processed 455,000 of 1,038,036 rows
```

```
Processed 456,000 of 1,038,036 rows
Processed 457,000 of 1,038,036 rows
Processed 458,000 of 1,038,036 rows
Processed 459,000 of 1,038,036 rows
Processed 460,000 of 1,038,036 rows
Processed 461,000 of 1,038,036 rows
Processed 462,000 of 1,038,036 rows
Processed 463,000 of 1,038,036 rows
Processed 464,000 of 1,038,036 rows
Processed 465,000 of 1,038,036 rows
Processed 466,000 of 1,038,036 rows
Processed 467,000 of 1,038,036 rows
Processed 468,000 of 1,038,036 rows
Processed 469,000 of 1,038,036 rows
Processed 470,000 of 1,038,036 rows
Processed 471,000 of 1,038,036 rows
Processed 472,000 of 1,038,036 rows
Processed 473,000 of 1,038,036 rows
Processed 474,000 of 1,038,036 rows
Processed 475,000 of 1,038,036 rows
Processed 476,000 of 1,038,036 rows
Processed 477,000 of 1,038,036 rows
Processed 478,000 of 1,038,036 rows
Processed 479,000 of 1,038,036 rows
Processed 480,000 of 1,038,036 rows
Processed 481,000 of 1,038,036 rows
Processed 482,000 of 1,038,036 rows
Processed 483,000 of 1,038,036 rows
Processed 484,000 of 1,038,036 rows
Processed 485,000 of 1,038,036 rows
Processed 486,000 of 1,038,036 rows
Processed 487,000 of 1,038,036 rows
Processed 488,000 of 1,038,036 rows
Processed 489,000 of 1,038,036 rows
Processed 490,000 of 1,038,036 rows
Processed 491,000 of 1,038,036 rows
Processed 492,000 of 1,038,036 rows
Processed 493,000 of 1,038,036 rows
Processed 494,000 of 1,038,036 rows
Processed 495,000 of 1,038,036 rows
Processed 496,000 of 1,038,036 rows
Processed 497,000 of 1,038,036 rows
Processed 498,000 of 1,038,036 rows
Processed 499,000 of 1,038,036 rows
Processed 500,000 of 1,038,036 rows
Processed 501,000 of 1,038,036 rows
Processed 502,000 of 1,038,036 rows
Processed 503,000 of 1,038,036 rows
Processed 504,000 of 1,038,036 rows
Processed 505,000 of 1,038,036 rows
```

```
Processed 506,000 of 1,038,036 rows
Processed 507,000 of 1,038,036 rows
Processed 508,000 of 1,038,036 rows
Processed 509,000 of 1,038,036 rows
Processed 510,000 of 1,038,036 rows
Processed 511,000 of 1,038,036 rows
Processed 512,000 of 1,038,036 rows
Processed 513,000 of 1,038,036 rows
Processed 514,000 of 1,038,036 rows
Processed 515,000 of 1,038,036 rows
Processed 516,000 of 1,038,036 rows
Processed 517,000 of 1,038,036 rows
Processed 518,000 of 1,038,036 rows
Processed 519,000 of 1,038,036 rows
Processed 520,000 of 1,038,036 rows
Processed 521,000 of 1,038,036 rows
Processed 522,000 of 1,038,036 rows
Processed 523,000 of 1,038,036 rows
Processed 524,000 of 1,038,036 rows
Processed 525,000 of 1,038,036 rows
Processed 526,000 of 1,038,036 rows
Processed 527,000 of 1,038,036 rows
Processed 528,000 of 1,038,036 rows
Processed 529,000 of 1,038,036 rows
Processed 530,000 of 1,038,036 rows
Processed 531,000 of 1,038,036 rows
Processed 532,000 of 1,038,036 rows
Processed 533,000 of 1,038,036 rows
Processed 534,000 of 1,038,036 rows
Processed 535,000 of 1,038,036 rows
Processed 536,000 of 1,038,036 rows
Processed 537,000 of 1,038,036 rows
Processed 538,000 of 1,038,036 rows
Processed 539,000 of 1,038,036 rows
Processed 540,000 of 1,038,036 rows
Processed 541,000 of 1,038,036 rows
Processed 542,000 of 1,038,036 rows
Processed 543,000 of 1,038,036 rows
Processed 544,000 of 1,038,036 rows
Processed 545,000 of 1,038,036 rows
Processed 546,000 of 1,038,036 rows
Processed 547,000 of 1,038,036 rows
Processed 548,000 of 1,038,036 rows
Processed 549,000 of 1,038,036 rows
Processed 550,000 of 1,038,036 rows
Processed 551,000 of 1,038,036 rows
Processed 552,000 of 1,038,036 rows
Processed 553,000 of 1,038,036 rows
Processed 554,000 of 1,038,036 rows
```

```
Processed 555,000 of 1,038,036 rows
Processed 556,000 of 1,038,036 rows
Processed 557,000 of 1,038,036 rows
Processed 558,000 of 1,038,036 rows
Processed 559,000 of 1,038,036 rows
Processed 560,000 of 1,038,036 rows
Processed 561,000 of 1,038,036 rows
Processed 562,000 of 1,038,036 rows
Processed 563,000 of 1,038,036 rows
Processed 564,000 of 1,038,036 rows
Processed 565,000 of 1,038,036 rows
Processed 566,000 of 1,038,036 rows
Processed 567,000 of 1,038,036 rows
Processed 568,000 of 1,038,036 rows
Processed 569,000 of 1,038,036 rows
Processed 570,000 of 1,038,036 rows
Processed 571,000 of 1,038,036 rows
Processed 572,000 of 1,038,036 rows
Processed 573,000 of 1,038,036 rows
Processed 574,000 of 1,038,036 rows
Processed 575,000 of 1,038,036 rows
Processed 576,000 of 1,038,036 rows
Processed 577,000 of 1,038,036 rows
Processed 578,000 of 1,038,036 rows
Processed 579,000 of 1,038,036 rows
Processed 580,000 of 1,038,036 rows
Processed 581,000 of 1,038,036 rows
Processed 582,000 of 1,038,036 rows
Processed 583,000 of 1,038,036 rows
Processed 584,000 of 1,038,036 rows
Processed 585,000 of 1,038,036 rows
Processed 586,000 of 1,038,036 rows
Processed 587,000 of 1,038,036 rows
Processed 588,000 of 1,038,036 rows
Processed 589,000 of 1,038,036 rows
Processed 590,000 of 1,038,036 rows
Processed 591,000 of 1,038,036 rows
Processed 592,000 of 1,038,036 rows
Processed 593,000 of 1,038,036 rows
Processed 594,000 of 1,038,036 rows
Processed 595,000 of 1,038,036 rows
Processed 596,000 of 1,038,036 rows
Processed 597,000 of 1,038,036 rows
Processed 598,000 of 1,038,036 rows
Processed 599,000 of 1,038,036 rows
Processed 600,000 of 1,038,036 rows
Processed 601,000 of 1,038,036 rows
Processed 602,000 of 1,038,036 rows
Processed 603,000 of 1,038,036 rows
```

```
Processed 604,000 of 1,038,036 rows
Processed 605,000 of 1,038,036 rows
Processed 606,000 of 1,038,036 rows
Processed 607,000 of 1,038,036 rows
Processed 608,000 of 1,038,036 rows
Processed 609,000 of 1,038,036 rows
Processed 610,000 of 1,038,036 rows
Processed 611,000 of 1,038,036 rows
Processed 612,000 of 1,038,036 rows
Processed 613,000 of 1,038,036 rows
Processed 614,000 of 1,038,036 rows
Processed 615,000 of 1,038,036 rows
Processed 616,000 of 1,038,036 rows
Processed 617,000 of 1,038,036 rows
Processed 618,000 of 1,038,036 rows
Processed 619,000 of 1,038,036 rows
Processed 620,000 of 1,038,036 rows
Processed 621,000 of 1,038,036 rows
Processed 622,000 of 1,038,036 rows
Processed 623,000 of 1,038,036 rows
Processed 624,000 of 1,038,036 rows
Processed 625,000 of 1,038,036 rows
Processed 626,000 of 1,038,036 rows
Processed 627,000 of 1,038,036 rows
Processed 628,000 of 1,038,036 rows
Processed 629,000 of 1,038,036 rows
Processed 630,000 of 1,038,036 rows
Processed 631,000 of 1,038,036 rows
Processed 632,000 of 1,038,036 rows
Processed 633,000 of 1,038,036 rows
Processed 634,000 of 1,038,036 rows
Processed 635,000 of 1,038,036 rows
Processed 636,000 of 1,038,036 rows
Processed 637,000 of 1,038,036 rows
Processed 638,000 of 1,038,036 rows
Processed 639,000 of 1,038,036 rows
Processed 640,000 of 1,038,036 rows
Processed 641,000 of 1,038,036 rows
Processed 642,000 of 1,038,036 rows
Processed 643,000 of 1,038,036 rows
Processed 644,000 of 1,038,036 rows
Processed 645,000 of 1,038,036 rows
Processed 646,000 of 1,038,036 rows
Processed 647,000 of 1,038,036 rows
Processed 648,000 of 1,038,036 rows
Processed 649,000 of 1,038,036 rows
Processed 650,000 of 1,038,036 rows
Processed 651,000 of 1,038,036 rows
Processed 652,000 of 1,038,036 rows
```

```
Processed 653,000 of 1,038,036 rows
Processed 654,000 of 1,038,036 rows
Processed 655,000 of 1,038,036 rows
Processed 656,000 of 1,038,036 rows
Processed 657,000 of 1,038,036 rows
Processed 658,000 of 1,038,036 rows
Processed 659,000 of 1,038,036 rows
Processed 660,000 of 1,038,036 rows
Processed 661,000 of 1,038,036 rows
Processed 662,000 of 1,038,036 rows
Processed 663,000 of 1,038,036 rows
Processed 664,000 of 1,038,036 rows
Processed 665,000 of 1,038,036 rows
Processed 666,000 of 1,038,036 rows
Processed 667,000 of 1,038,036 rows
Processed 668,000 of 1,038,036 rows
Processed 669,000 of 1,038,036 rows
Processed 670,000 of 1,038,036 rows
Processed 671,000 of 1,038,036 rows
Processed 672,000 of 1,038,036 rows
Processed 673,000 of 1,038,036 rows
Processed 674,000 of 1,038,036 rows
Processed 675,000 of 1,038,036 rows
Processed 676,000 of 1,038,036 rows
Processed 677,000 of 1,038,036 rows
Processed 678,000 of 1,038,036 rows
Processed 679,000 of 1,038,036 rows
Processed 680,000 of 1,038,036 rows
Processed 681,000 of 1,038,036 rows
Processed 682,000 of 1,038,036 rows
Processed 683,000 of 1,038,036 rows
Processed 684,000 of 1,038,036 rows
Processed 685,000 of 1,038,036 rows
Processed 686,000 of 1,038,036 rows
Processed 687,000 of 1,038,036 rows
Processed 688,000 of 1,038,036 rows
Processed 689,000 of 1,038,036 rows
Processed 690,000 of 1,038,036 rows
Processed 691,000 of 1,038,036 rows
Processed 692,000 of 1,038,036 rows
Processed 693,000 of 1,038,036 rows
Processed 694,000 of 1,038,036 rows
Processed 695,000 of 1,038,036 rows
Processed 696,000 of 1,038,036 rows
Processed 697,000 of 1,038,036 rows
Processed 698,000 of 1,038,036 rows
Processed 699,000 of 1,038,036 rows
Processed 700,000 of 1,038,036 rows
Processed 701,000 of 1,038,036 rows
```

```
Processed 702,000 of 1,038,036 rows
Processed 703,000 of 1,038,036 rows
Processed 704,000 of 1,038,036 rows
Processed 705,000 of 1,038,036 rows
Processed 706,000 of 1,038,036 rows
Processed 707,000 of 1,038,036 rows
Processed 708,000 of 1,038,036 rows
Processed 709,000 of 1,038,036 rows
Processed 710,000 of 1,038,036 rows
Processed 711,000 of 1,038,036 rows
Processed 712,000 of 1,038,036 rows
Processed 713,000 of 1,038,036 rows
Processed 714,000 of 1,038,036 rows
Processed 715,000 of 1,038,036 rows
Processed 716,000 of 1,038,036 rows
Processed 717,000 of 1,038,036 rows
Processed 718,000 of 1,038,036 rows
Processed 719,000 of 1,038,036 rows
Processed 720,000 of 1,038,036 rows
Processed 721,000 of 1,038,036 rows
Processed 722,000 of 1,038,036 rows
Processed 723,000 of 1,038,036 rows
Processed 724,000 of 1,038,036 rows
Processed 725,000 of 1,038,036 rows
Processed 726,000 of 1,038,036 rows
Processed 727,000 of 1,038,036 rows
Processed 728,000 of 1,038,036 rows
Processed 729,000 of 1,038,036 rows
Processed 730,000 of 1,038,036 rows
Processed 731,000 of 1,038,036 rows
Processed 732,000 of 1,038,036 rows
Processed 733,000 of 1,038,036 rows
Processed 734,000 of 1,038,036 rows
Processed 735,000 of 1,038,036 rows
Processed 736,000 of 1,038,036 rows
Processed 737,000 of 1,038,036 rows
Processed 738,000 of 1,038,036 rows
Processed 739,000 of 1,038,036 rows
Processed 740,000 of 1,038,036 rows
Processed 741,000 of 1,038,036 rows
Processed 742,000 of 1,038,036 rows
Processed 743,000 of 1,038,036 rows
Processed 744,000 of 1,038,036 rows
Processed 745,000 of 1,038,036 rows
Processed 746,000 of 1,038,036 rows
Processed 747,000 of 1,038,036 rows
Processed 748,000 of 1,038,036 rows
Processed 749,000 of 1,038,036 rows
Processed 750,000 of 1,038,036 rows
```

```
Processed 751,000 of 1,038,036 rows
Processed 752,000 of 1,038,036 rows
Processed 753,000 of 1,038,036 rows
Processed 754,000 of 1,038,036 rows
Processed 755,000 of 1,038,036 rows
Processed 756,000 of 1,038,036 rows
Processed 757,000 of 1,038,036 rows
Processed 758,000 of 1,038,036 rows
Processed 759,000 of 1,038,036 rows
Processed 760,000 of 1,038,036 rows
Processed 761,000 of 1,038,036 rows
Processed 762,000 of 1,038,036 rows
Processed 763,000 of 1,038,036 rows
Processed 764,000 of 1,038,036 rows
Processed 765,000 of 1,038,036 rows
Processed 766,000 of 1,038,036 rows
Processed 767,000 of 1,038,036 rows
Processed 768,000 of 1,038,036 rows
Processed 769,000 of 1,038,036 rows
Processed 770,000 of 1,038,036 rows
Processed 771,000 of 1,038,036 rows
Processed 772,000 of 1,038,036 rows
Processed 773,000 of 1,038,036 rows
Processed 774,000 of 1,038,036 rows
Processed 775,000 of 1,038,036 rows
Processed 776,000 of 1,038,036 rows
Processed 777,000 of 1,038,036 rows
Processed 778,000 of 1,038,036 rows
Processed 779,000 of 1,038,036 rows
Processed 780,000 of 1,038,036 rows
Processed 781,000 of 1,038,036 rows
Processed 782,000 of 1,038,036 rows
Processed 783,000 of 1,038,036 rows
Processed 784,000 of 1,038,036 rows
Processed 785,000 of 1,038,036 rows
Processed 786,000 of 1,038,036 rows
Processed 787,000 of 1,038,036 rows
Processed 788,000 of 1,038,036 rows
Processed 789,000 of 1,038,036 rows
Processed 790,000 of 1,038,036 rows
Processed 791,000 of 1,038,036 rows
Processed 792,000 of 1,038,036 rows
Processed 793,000 of 1,038,036 rows
Processed 794,000 of 1,038,036 rows
Processed 795,000 of 1,038,036 rows
Processed 796,000 of 1,038,036 rows
Processed 797,000 of 1,038,036 rows
Processed 798,000 of 1,038,036 rows
Processed 799,000 of 1,038,036 rows
```

```
Processed 800,000 of 1,038,036 rows
Processed 801,000 of 1,038,036 rows
Processed 802,000 of 1,038,036 rows
Processed 803,000 of 1,038,036 rows
Processed 804,000 of 1,038,036 rows
Processed 805,000 of 1,038,036 rows
Processed 806,000 of 1,038,036 rows
Processed 807,000 of 1,038,036 rows
Processed 808,000 of 1,038,036 rows
Processed 809,000 of 1,038,036 rows
Processed 810,000 of 1,038,036 rows
Processed 811,000 of 1,038,036 rows
Processed 812,000 of 1,038,036 rows
Processed 813,000 of 1,038,036 rows
Processed 814,000 of 1,038,036 rows
Processed 815,000 of 1,038,036 rows
Processed 816,000 of 1,038,036 rows
Processed 817,000 of 1,038,036 rows
Processed 818,000 of 1,038,036 rows
Processed 819,000 of 1,038,036 rows
Processed 820,000 of 1,038,036 rows
Processed 821,000 of 1,038,036 rows
Processed 822,000 of 1,038,036 rows
Processed 823,000 of 1,038,036 rows
Processed 824,000 of 1,038,036 rows
Processed 825,000 of 1,038,036 rows
Processed 826,000 of 1,038,036 rows
Processed 827,000 of 1,038,036 rows
Processed 828,000 of 1,038,036 rows
Processed 829,000 of 1,038,036 rows
Processed 830,000 of 1,038,036 rows
Processed 831,000 of 1,038,036 rows
Processed 832,000 of 1,038,036 rows
Processed 833,000 of 1,038,036 rows
Processed 834,000 of 1,038,036 rows
Processed 835,000 of 1,038,036 rows
Processed 836,000 of 1,038,036 rows
Processed 837,000 of 1,038,036 rows
Processed 838,000 of 1,038,036 rows
Processed 839,000 of 1,038,036 rows
Processed 840,000 of 1,038,036 rows
Processed 841,000 of 1,038,036 rows
Processed 842,000 of 1,038,036 rows
Processed 843,000 of 1,038,036 rows
Processed 844,000 of 1,038,036 rows
Processed 845,000 of 1,038,036 rows
Processed 846,000 of 1,038,036 rows
Processed 847,000 of 1,038,036 rows
Processed 848,000 of 1,038,036 rows
```

```
Processed 849,000 of 1,038,036 rows
Processed 850,000 of 1,038,036 rows
Processed 851,000 of 1,038,036 rows
Processed 852,000 of 1,038,036 rows
Processed 853,000 of 1,038,036 rows
Processed 854,000 of 1,038,036 rows
Processed 855,000 of 1,038,036 rows
Processed 856,000 of 1,038,036 rows
Processed 857,000 of 1,038,036 rows
Processed 858,000 of 1,038,036 rows
Processed 859,000 of 1,038,036 rows
Processed 860,000 of 1,038,036 rows
Processed 861,000 of 1,038,036 rows
Processed 862,000 of 1,038,036 rows
Processed 863,000 of 1,038,036 rows
Processed 864,000 of 1,038,036 rows
Processed 865,000 of 1,038,036 rows
Processed 866,000 of 1,038,036 rows
Processed 867,000 of 1,038,036 rows
Processed 868,000 of 1,038,036 rows
Processed 869,000 of 1,038,036 rows
Processed 870,000 of 1,038,036 rows
Processed 871,000 of 1,038,036 rows
Processed 872,000 of 1,038,036 rows
Processed 873,000 of 1,038,036 rows
Processed 874,000 of 1,038,036 rows
Processed 875,000 of 1,038,036 rows
Processed 876,000 of 1,038,036 rows
Processed 877,000 of 1,038,036 rows
Processed 878,000 of 1,038,036 rows
Processed 879,000 of 1,038,036 rows
Processed 880,000 of 1,038,036 rows
Processed 881,000 of 1,038,036 rows
Processed 882,000 of 1,038,036 rows
Processed 883,000 of 1,038,036 rows
Processed 884,000 of 1,038,036 rows
Processed 885,000 of 1,038,036 rows
Processed 886,000 of 1,038,036 rows
Processed 887,000 of 1,038,036 rows
Processed 888,000 of 1,038,036 rows
Processed 889,000 of 1,038,036 rows
Processed 890,000 of 1,038,036 rows
Processed 891,000 of 1,038,036 rows
Processed 892,000 of 1,038,036 rows
Processed 893,000 of 1,038,036 rows
Processed 894,000 of 1,038,036 rows
Processed 895,000 of 1,038,036 rows
Processed 896,000 of 1,038,036 rows
Processed 897,000 of 1,038,036 rows
```

```
Processed 898,000 of 1,038,036 rows
Processed 899,000 of 1,038,036 rows
Processed 900,000 of 1,038,036 rows
Processed 901,000 of 1,038,036 rows
Processed 902,000 of 1,038,036 rows
Processed 903,000 of 1,038,036 rows
Processed 904,000 of 1,038,036 rows
Processed 905,000 of 1,038,036 rows
Processed 906,000 of 1,038,036 rows
Processed 907,000 of 1,038,036 rows
Processed 908,000 of 1,038,036 rows
Processed 909,000 of 1,038,036 rows
Processed 910,000 of 1,038,036 rows
Processed 911,000 of 1,038,036 rows
Processed 912,000 of 1,038,036 rows
Processed 913,000 of 1,038,036 rows
Processed 914,000 of 1,038,036 rows
Processed 915,000 of 1,038,036 rows
Processed 916,000 of 1,038,036 rows
Processed 917,000 of 1,038,036 rows
Processed 918,000 of 1,038,036 rows
Processed 919,000 of 1,038,036 rows
Processed 920,000 of 1,038,036 rows
Processed 921,000 of 1,038,036 rows
Processed 922,000 of 1,038,036 rows
Processed 923,000 of 1,038,036 rows
Processed 924,000 of 1,038,036 rows
Processed 925,000 of 1,038,036 rows
Processed 926,000 of 1,038,036 rows
Processed 927,000 of 1,038,036 rows
Processed 928,000 of 1,038,036 rows
Processed 929,000 of 1,038,036 rows
Processed 930,000 of 1,038,036 rows
Processed 931,000 of 1,038,036 rows
Processed 932,000 of 1,038,036 rows
Processed 933,000 of 1,038,036 rows
Processed 934,000 of 1,038,036 rows
Processed 935,000 of 1,038,036 rows
Processed 936,000 of 1,038,036 rows
Processed 937,000 of 1,038,036 rows
Processed 938,000 of 1,038,036 rows
Processed 939,000 of 1,038,036 rows
Processed 940,000 of 1,038,036 rows
Processed 941,000 of 1,038,036 rows
Processed 942,000 of 1,038,036 rows
Processed 943,000 of 1,038,036 rows
Processed 944,000 of 1,038,036 rows
Processed 945,000 of 1,038,036 rows
Processed 946,000 of 1,038,036 rows
```

```
Processed 947,000 of 1,038,036 rows
Processed 948,000 of 1,038,036 rows
Processed 949,000 of 1,038,036 rows
Processed 950,000 of 1,038,036 rows
Processed 951,000 of 1,038,036 rows
Processed 952,000 of 1,038,036 rows
Processed 953,000 of 1,038,036 rows
Processed 954,000 of 1,038,036 rows
Processed 955,000 of 1,038,036 rows
Processed 956,000 of 1,038,036 rows
Processed 957,000 of 1,038,036 rows
Processed 958,000 of 1,038,036 rows
Processed 959,000 of 1,038,036 rows
Processed 960,000 of 1,038,036 rows
Processed 961,000 of 1,038,036 rows
Processed 962,000 of 1,038,036 rows
Processed 963,000 of 1,038,036 rows
Processed 964,000 of 1,038,036 rows
Processed 965,000 of 1,038,036 rows
Processed 966,000 of 1,038,036 rows
Processed 967,000 of 1,038,036 rows
Processed 968,000 of 1,038,036 rows
Processed 969,000 of 1,038,036 rows
Processed 970,000 of 1,038,036 rows
Processed 971,000 of 1,038,036 rows
Processed 972,000 of 1,038,036 rows
Processed 973,000 of 1,038,036 rows
Processed 974,000 of 1,038,036 rows
Processed 975,000 of 1,038,036 rows
Processed 976,000 of 1,038,036 rows
Processed 977,000 of 1,038,036 rows
Processed 978,000 of 1,038,036 rows
Processed 979,000 of 1,038,036 rows
Processed 980,000 of 1,038,036 rows
Processed 981,000 of 1,038,036 rows
Processed 982,000 of 1,038,036 rows
Processed 983,000 of 1,038,036 rows
Processed 984,000 of 1,038,036 rows
Processed 985,000 of 1,038,036 rows
Processed 986,000 of 1,038,036 rows
Processed 987,000 of 1,038,036 rows
Processed 988,000 of 1,038,036 rows
Processed 989,000 of 1,038,036 rows
Processed 990,000 of 1,038,036 rows
Processed 991,000 of 1,038,036 rows
Processed 992,000 of 1,038,036 rows
Processed 993,000 of 1,038,036 rows
Processed 994,000 of 1,038,036 rows
Processed 995,000 of 1,038,036 rows
```

```
Processed 996,000 of 1,038,036 rows
Processed 997,000 of 1,038,036 rows
Processed 998,000 of 1,038,036 rows
Processed 999,000 of 1,038,036 rows
Processed 1,000,000 of 1,038,036 rows
Processed 1,001,000 of 1,038,036 rows
Processed 1,002,000 of 1,038,036 rows
Processed 1,003,000 of 1,038,036 rows
Processed 1,004,000 of 1,038,036 rows
Processed 1,005,000 of 1,038,036 rows
Processed 1,006,000 of 1,038,036 rows
Processed 1,007,000 of 1,038,036 rows
Processed 1,008,000 of 1,038,036 rows
Processed 1,009,000 of 1,038,036 rows
Processed 1,010,000 of 1,038,036 rows
Processed 1,011,000 of 1,038,036 rows
Processed 1,012,000 of 1,038,036 rows
Processed 1,013,000 of 1,038,036 rows
Processed 1,014,000 of 1,038,036 rows
Processed 1,015,000 of 1,038,036 rows
Processed 1,016,000 of 1,038,036 rows
Processed 1,017,000 of 1,038,036 rows
Processed 1,018,000 of 1,038,036 rows
Processed 1,019,000 of 1,038,036 rows
Processed 1,020,000 of 1,038,036 rows
Processed 1,021,000 of 1,038,036 rows
Processed 1,022,000 of 1,038,036 rows
Processed 1,023,000 of 1,038,036 rows
Processed 1,024,000 of 1,038,036 rows
Processed 1,025,000 of 1,038,036 rows
Processed 1,026,000 of 1,038,036 rows
Processed 1,027,000 of 1,038,036 rows
Processed 1,028,000 of 1,038,036 rows
Processed 1,029,000 of 1,038,036 rows
Processed 1,030,000 of 1,038,036 rows
Processed 1,031,000 of 1,038,036 rows
Processed 1,032,000 of 1,038,036 rows
Processed 1,033,000 of 1,038,036 rows
Processed 1,034,000 of 1,038,036 rows
Processed 1,035,000 of 1,038,036 rows
Processed 1,036,000 of 1,038,036 rows
Processed 1,037,000 of 1,038,036 rows
Processed 1,038,000 of 1,038,036 rows
Processed 1,038,036 of 1,038,036 rows
Upload finished: 1,038,036 records processed.
```

```
#index to find each observation during upload
#since i needed to run the above cell again
collection.create_index(
    [("connection", 1), ("direction", 1), ("time", 1)],
```

```
    unique=True,
)
print("Index created.")
```

```
Index created.
```

PART 3

```
import geopandas as gdp

#different regions of Norway
gdf_region = gdp.read_file(r"C:\Users\malki\OneDrive - Norwegian
University of Life Sciences\UNI5\IND320-1\Project\data\NVE_Region\
NVE_Region.shp")

gdf_region
```

```
   OBJECTID REGION         REGIONNAVN POSTNUMMER   POSTSTED  \
0       1.0     RV        Region Vest       6801     FÃ¸rde
1       2.0     RM  Region Midt-Norge       7030  Trondheim
2       3.0     RN        Region Nord       8514     Narvik
3       4.0    RÃ        Region Ã□st       2307      Hamar
4       5.0     RS        Region SÃ¸r       3103  TÃ¸nsberg

                BESÃ□KSADR            ADRESSE  \
0       Naustdalsveien 1B        Postboks 53
1  Abelsgate 9, Trondheim        Abelsgate 9
2       Kongensgate 52-54  Kongensgate 52-54
3           Vangsvegen 73      Postboks 4223
4   Anton Jenssens gate 7      Postboks 2124

                                             geometry
0  POLYGON ((193105.76 6929797.839, 191705.75 692...
1  POLYGON ((391250 7388620, 392270 7386790, 3937...
2  POLYGON ((1109096.413 7844647.502, 1105115.343...
3  POLYGON ((253748.22 6678130.91, 253645.58 6678...
4  POLYGON ((253748.22 6678130.91, 253917.63 6677...
```

```
#relabeling the regions
gdf_region["REGION"] = gdf_region["REGION"].replace({
    "RV": "West",
    "RM": "North",
    "RN": "North",
    "RS": "East",
    "R\u00c3\u0098": "East",
})

gdf_region
```

```
   OBJECTID REGION         REGIONNAVN POSTNUMMER   POSTSTED  \
0       1.0   West        Region Vest       6801     FÃ¸rde
1       2.0  North  Region Midt-Norge       7030  Trondheim
2       3.0  North        Region Nord       8514     Narvik
3       4.0   East        Region Ã□st       2307      Hamar
4       5.0   East        Region SÃ¸r       3103  TÃ¸nsberg

                BESÃ□KSADR            ADRESSE  \
0       Naustdalsveien 1B        Postboks 53
1  Abelsgate 9, Trondheim        Abelsgate 9
2       Kongensgate 52-54  Kongensgate 52-54
3           Vangsvegen 73      Postboks 4223
4   Anton Jenssens gate 7      Postboks 2124

                                              geometry
0  POLYGON ((193105.76 6929797.839, 191705.75 692...
1  POLYGON ((391250 7388620, 392270 7386790, 3937...
2  POLYGON ((1109096.413 7844647.502, 1105115.343...
3  POLYGON ((253748.22 6678130.91, 253645.58 6678...
4  POLYGON ((253748.22 6678130.91, 253917.63 6677...
```

```
regions = gdf_region[["REGION", "geometry"]].dissolve(by="REGION")
#combining the right polygons
regions
```

```
                                                 geometry
REGION
East    POLYGON ((244204.94 6521981.68, 231754.99 6523...
North   POLYGON ((522969.98 7381455.32, 520050.96 7378...
West    POLYGON ((193105.76 6929797.839, 191705.75 692...
```

```
sampled_locations = regions.geometry.sample_points(size=1000)
#sampling 1000 samples for each region
len(sampled_locations.explode()) #checking that 3000 samples are
collected
```

```
3000
```

```
print(sampled_locations.crs)
```

```
EPSG:25833
```

```
sampled_locations = sampled_locations.to_crs("EPSG:4326")
#reprojecting the sampled locations

#electricity areas
gdf_elarea = gdp.read_file(r"C:\Users\malki\OneDrive - Norwegian
University of Life Sciences\UNI5\IND320-1\Project\data\Budområder\
Budområder.shp")
gdf_elarea
```

```
   objectid budomr  globalid    st_area(sh    st_perimet  \
0       1.0   NO 1       NaN  6.052902e+10  1.657761e+06
1       2.0   NO 2       NaN  6.875969e+10  1.256473e+06
2       3.0   NO 3       NaN  1.042199e+11  1.773323e+06
3       4.0   NO 4       NaN  2.032500e+11  3.385499e+06
4       5.0   NO 5       NaN  3.334870e+10  1.106762e+06

                                             geometry
0  POLYGON ((337473.63 6970542.97, 337711.9 69703...
1  POLYGON ((-9833.91 6726499.86, -9836 6726404.0...
2  POLYGON ((308588.7 7261519.43, 313576.63 72590...
3  POLYGON ((973920.93 7962739.19, 974922.77 7962...
4  POLYGON ((101466.43 6881318.54, 101525.43 6881...
```

```
sampled_elarea = gdf_elarea.geometry.sample_points(size=1000)
#sampling 1000 samples for each elarea
len(sampled_elarea.explode()) #checking that 5000 samples are
collected
```

```
5000
```

```
(sampled_elarea[gdf_elarea["budomr"] == "NO 1"]).explode() #just
looking at the sampled points for NO1
```

```
0    POINT (315097.624 6818382.302)
0    POINT (271011.759 6761141.758)
0    POINT (331487.718 6767571.655)
0    POINT (310478.274 6907532.405)
0    POINT (282296.531 6845105.272)
                 ...
0    POINT (319200.359 6944206.493)
0     POINT (302168.27 6897839.844)
0    POINT (230210.509 6675667.225)
0    POINT (250224.233 6885756.708)
0    POINT (346008.496 6683920.462)
Name: sampled_points, Length: 1000, dtype: geometry
```

```
print(sampled_elarea.crs) #checking the coordinate reference system of
the sampled electricity areas
```

```
EPSG:25833
```

EPSG: 25833 means that the coordinates are in metres, but we need them in latitude and longitude. This is equivalengt to ESPG: 4326.

```
sampled_elarea = sampled_elarea.to_crs("EPSG:4326") #reprojecting the
sampled electricity areas

(sampled_elarea[gdf_elarea["budomr"] == "NO 1"]).explode() #checking
that the reprojecting worked
```

```
0    POINT (11.53045 61.45494)
0    POINT (10.77494 60.91894)
0    POINT (11.88286 61.00694)
0    POINT (11.35011 62.25141)
0    POINT (10.88506 61.67719)
               ...
0    POINT (11.47927 62.58429)
0     POINT (11.20141 62.1603)
0    POINT (10.14133 60.12865)
0    POINT (10.22478 62.02134)
0    POINT (12.21654 60.26273)
Name: sampled_points, Length: 1000, dtype: geometry
```

```
#sampling a random point from the electricity area NO3
no3_points = sampled_elarea[
    gdf_elarea["budomr"] == "NO 3"].explode().to_crs(epsg=4326)

rdn_pnt_no3 = no3_points.sample(n=1).iloc[0]

longitude = rdn_pnt_no3.x
latitude = rdn_pnt_no3.y

longitude, latitude
```

```
(6.717619358640041, 62.943195720776416)
```

```
pd.DataFrame([{
    "Area_type": "EL",
    "Area_number": 3,
    "latitude": latitude,
    "longitude": longitude,
}]).to_csv("weather_locations.csv", index=False)

#fecthing weather data from the open-meteo API
def fetch_weather(latitude, longitude, start_date, end_date):
    response = requests.get(
        "https://archive-api.open-meteo.com/v1/archive",
        params={
            "latitude": latitude,
            "longitude": longitude,
            "start_date": start_date,
            "end_date": end_date,
            "models": "era5",
            "hourly": "temperature_2m,precipitation,snowfall",
            "timezone": "UTC",
        },
        timeout=60,
    )
    response.raise_for_status()
    return response.json()
```

```
#use the selected random point to fetch weather data for the year 2024
result = fetch_weather(latitude, longitude, "2024-01-01", "2024-12-
31")
weather = pd.DataFrame(result["hourly"])

weather.head() #looking at the first five days of 2024
```

	time	temperature_2m	precipitation	snowfall
0	2024-01-01T00:00	1.4	0.0	0.0
1	2024-01-01T01:00	1.2	0.0	0.0
2	2024-01-01T02:00	1.0	0.0	0.0
3	2024-01-01T03:00	0.9	0.0	0.0
4	2024-01-01T04:00	0.8	0.0	0.0

```
weather.isnull().sum() #checking for missing values/measures
```

```
time              0
temperature_2m    0
precipitation     0
snowfall          0
dtype: int64
```

Average temperature over time, outliers by DCT/SPC

We want to know if a day was unusally warm or cold for that time of the year. A temperature of 18 degrees celsius might be a normal temperature in the summer but unusual in the winter. So we need to remove the slow seasonal pattern before deciding what counts as unusual.

```
def temperature_outliers(weather, cutoff=6, n_std=3):
    # Calculate daily average temperature from the full year
    # group by date and take the mean of the temperature_2m column
    data = weather.copy()
    data["time"] = pd.to_datetime(data["time"])
    temperature = (
        data.set_index("time")["temperature_2m"]
        .resample("D").mean()
    )

    #Transform temperature into DCT coefficients
    #first convert the temperature series to a numpy array
    y = temperature.to_numpy()
    fourier_signal = dct(y, type=1, norm="forward")

    # Remove low-frequency components to obtain SATV
    W = np.arange(len(y)) #numbers the cofficients from 0 to the
length of the temperature series
    filtered_fourier_signal = fourier_signal.copy()
    filtered_fourier_signal[W < cutoff] = 0 #sets the low-frequency
components to zero, the days are not removed
    satv = idct(filtered_fourier_signal, type=1, norm="forward")
```

```
#remaining coefficients back to a value for each day

    #robust median and std estimated from the whole year
    median = np.median(satv) #gives the typical variation
    std = stats.median_abs_deviation(satv, scale="normal") #how far
the typical variation spreads from the median

    #add the seasonal pattern back to express boundaries in degrees
Celsius
    #the seasonal pattern is the difference between the original
temperature and the SATV
    seasonal = y - satv
    #calculate the lower and upper boundaries for outlier detection
    lower = seasonal + median - n_std * std
    upper = seasonal + median + n_std * std

    #Identify observations outside the boundaries
    is_outlier = np.abs(satv - median) > n_std * std
    outliers = temperature[is_outlier].to_frame(name="temperature_2m")

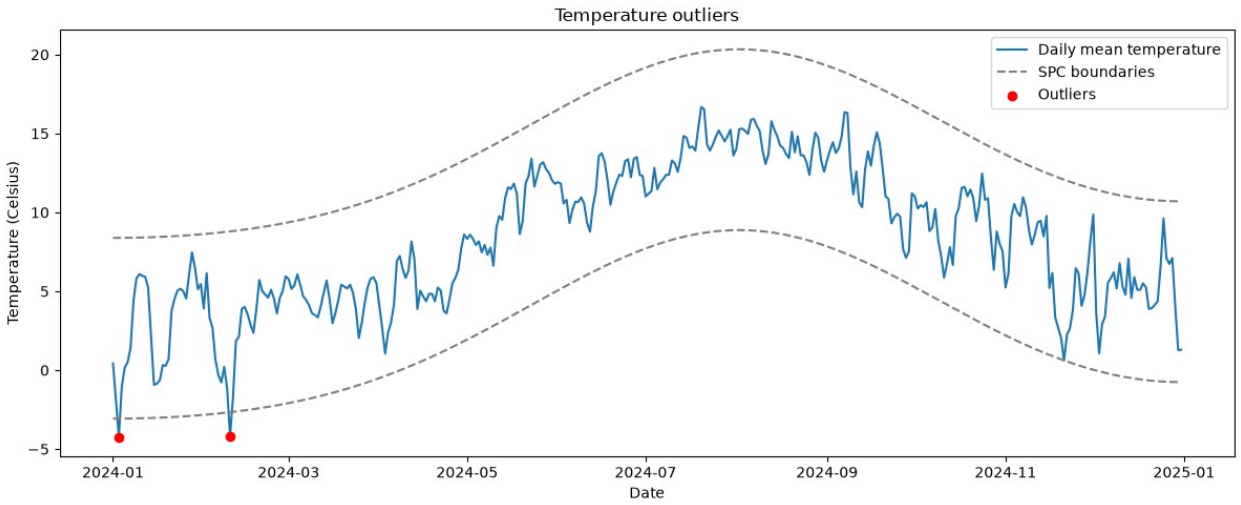
    # Plot original temperatures, boundaries and contrasting outlier
points.
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.plot(temperature.index, y, label="Daily mean temperature")
    ax.plot(temperature.index, lower, "--", color="grey",
            label="SPC boundaries")
    ax.plot(temperature.index, upper, "--", color="grey")
    ax.scatter(outliers.index, outliers["temperature_2m"],
               color="red", label="Outliers", zorder=3)
    ax.set_xlabel("Date")
    ax.set_ylabel("Temperature (Celsius)")
    ax.set_title("Temperature outliers")
    ax.legend()
    fig.tight_layout()

    summary = {
        "Number of outliers": len(outliers),
        "Percentage of outliers": round(100 * is_outlier.mean(), 2),
#how many percent of the days are outliers
        "Robust standard deviation": std,
    }

    return fig, summary, outliers

#Test the function on the downloaded weather data
fig, summary, outliers = temperature_outliers(weather)

plt.show()
print(summary)
display(outliers)
```




```
{'Number of outliers': 2, 'Percentage of outliers': 0.55, 'Robust
standard deviation': 1.9092947292528182}

            temperature_2m
time
2024-01-03       -4.225000
2024-02-10       -4.166667
```

We observe two outliers in the average temperature of the year 2024. Two days had average temperatures outside of the boundaries that is defined and visualized by the dotted lines.

Average temperature over time, outliers by LOF

```
def temperature_outliers_lof(weather, contamination=0.01,
n_neighbors=12):
    # Calculate daily average temperature
    data = weather.copy()
    data["time"] = pd.to_datetime(data["time"])
    temperature = (
        data.set_index("time")["temperature_2m"]
        .resample("D").mean()
        .dropna()
    )

    #Use Local Outlier Factor (LOF) to detect outliers in the daily
temperature data
    #contamination is the expected proportion of outliers in the data
    lof = LocalOutlierFactor(
        n_neighbors=n_neighbors,
        contamination=contamination,
    )
    pred_labels = lof.fit_predict(temperature.to_frame())

    # LOF labels outliers as -1 and inliers as 1.
```

```
    outliers = temperature[pred_labels == -1].to_frame(
        name="temperature_2m"
    )

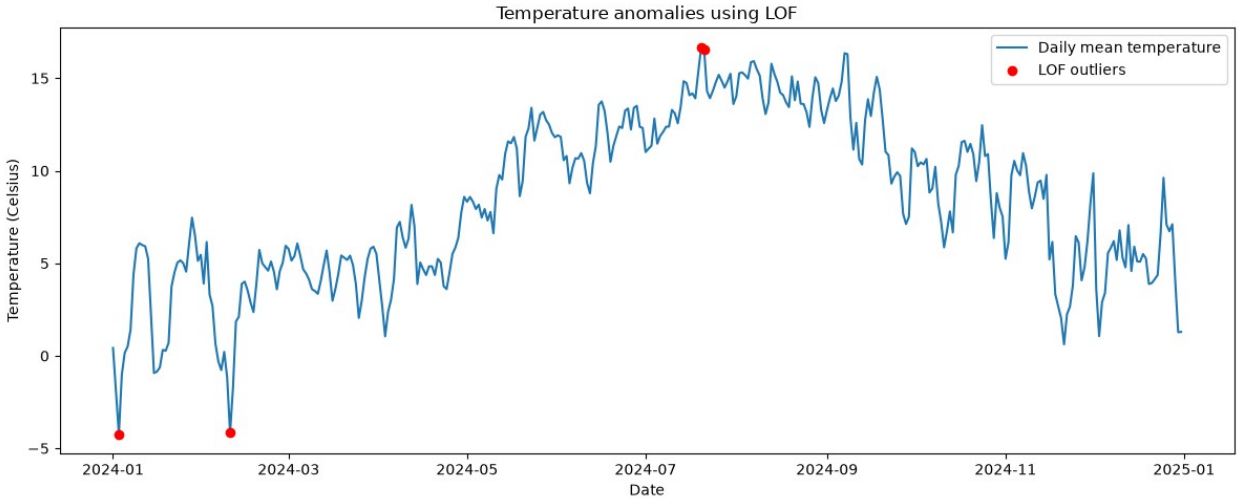
    # Plot daily temperatures and highlight the flagged days.
    fig, ax = plt.subplots(figsize=(12, 5))
    ax.plot(temperature.index, temperature, label="Daily mean
temperature")
    ax.scatter(
        outliers.index, outliers["temperature_2m"],
        color="red", label="LOF outliers", zorder=3,
    )
    ax.set_xlabel("Date")
    ax.set_ylabel("Temperature (Celsius)")
    ax.set_title("Temperature anomalies using LOF")
    ax.legend()
    fig.tight_layout()

    summary = {
        "Number of outliers": len(outliers),
        "Percentage of outliers": round(100 * outliers.shape[0]
                                        / temperature.shape[0], 2)
    }

    return fig, summary, outliers

#Test the function on the downloaded weather data
fig, summary, outliers = temperature_outliers_lof(weather)

plt.show()
print(summary)
display(outliers)
```




```
{'Number of outliers': 4, 'Percentage of outliers': 1.09}
```

```
            temperature_2m
time
2024-01-03       -4.225000
2024-02-10       -4.166667
2024-07-20       16.675000
2024-07-21       16.541667
```

Outlier detection by LOF detects the same two outliers as using DCT, in addition to two more days in the month of July, where the temperature seem to have been to high. If i set the contamination=0.005, the only outliers become the days in July. LOF uses a cutoff-method instead of boundaries, where this cutoff is adjusted for each day based on the neighboors. Adjusting the n_neighboors gave the same result from 9-30. I found 12 the most logical number as there are 12 months in a year.

STL on degree of filling

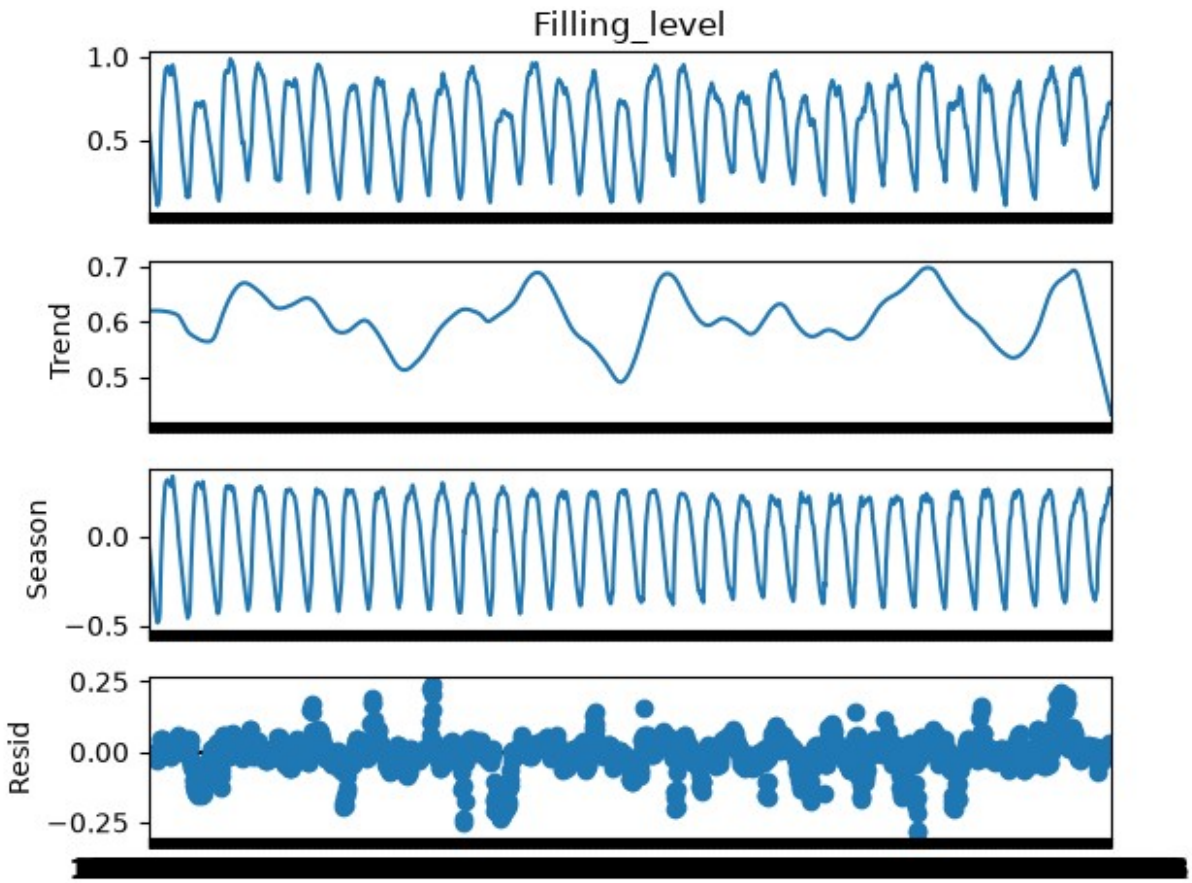
```
#selecting data from magasinstatistikk corresponding to NO3
el_3 = el_area[(el_area["Area_number"] == 3)].sort_values("Date")
filling = el_3.set_index("Date")["Filling_level"]

def plot_stl(filling, seasonal=7, trend=105, robust=True):
    #trend is set to 2 years
    #robust reduces influence of unusually high or low observations

    #decompose weekly filling levels using an annual cycle
    stl = STL(
        filling,
        period=52, #since we have weekly data, 52 weeks in a year
        seasonal=seasonal,
        trend=trend,
        robust=robust,
    )
    result = stl.fit()

    #plotting the original filling levels, trend, seasonal pattern and
residuals.
    fig = result.plot()
    fig.tight_layout()
    return fig

fig = plot_stl(filling)
plt.show()
```



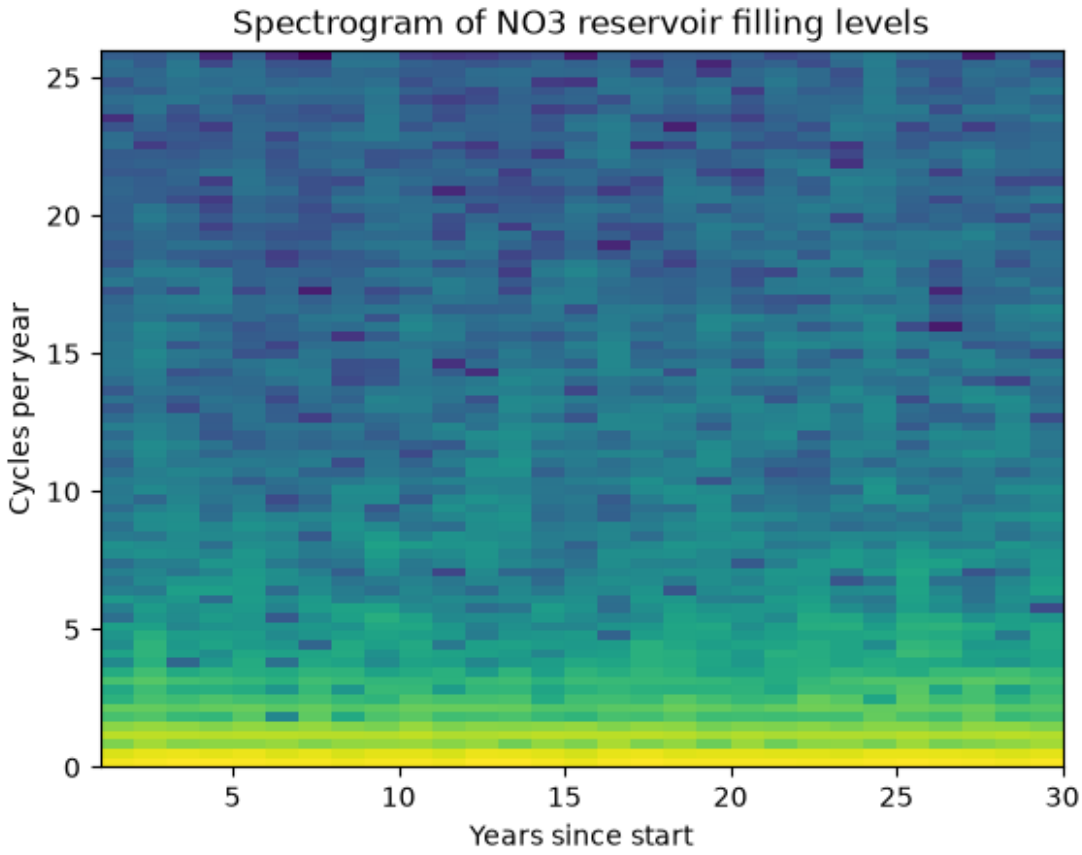
By looking at the "Season"-plot we see that STL captures the strong yearly pattern in reservoir filling levels for EL3 (corresponding to NO3). The residuals show clusters of positive and negative values, especially several negative dips. This indicates that some variation remains unexplained by the trend and seasonal pattern. By changing seasonal and trend, we observe that there becomes a clear pattern in the residuals, indicating that the STL didnt properly capture the patterns in the filling levels.

Spectrogram

```
def plot_spectrogram(filling, window_length=156, window_overlap=104):
    #each window covers 156 weeks=3 years
    #windows share 104 weeks=2 years of data
    #each new windows starts 52 weeks after the previous window
    fig, ax = plt.subplots()
    ax.specgram(
        filling,
        NFFT=window_length, #number of weekly observations in each
window
        noverlap=window_overlap, #observations shared between windows
        Fs=52, #52 observations per year, since we have weekly data
    )
    ax.set_xlabel("Years since start")
```

```
    ax.set_ylabel("Cycles per year")
    ax.set_title("Spectrogram of NO3 reservoir filling levels")
    return fig

#the same weekly filling data used for STL
fig = plot_spectrogram(filling)
plt.show()
```



By this spectogram we observe that the patterns are the strongest near the bottom, closer to zero cycles. That means the changes in reservoir filling happen slowly over months. Higher up, the colors are mostly blue and purple, indicating that fast rises and falls in the reservoir filling, is a weaker pattern (doesnt happen much). The colors are similar from left to right, suggesting that slow changes dominate throughtout the years.

Log

I found Part 3 simpler than Part 2. I already had a Streamlit app to build on, so I could spend more time working with the data and understanding the plots. Some of the methods were new to me, but I found it easier to work through them one at a time in the notebook.

One of the first things I had to understand was the difference between precipitation regions and electricity areas. I combined the precipitation regions into East, North and West, while NO3 came from a separate shapefile for electricity areas. At first, I was confused about how Open-

Meteo would know which area a point belonged to. It made sense once I realised that the shapefile tells me where the point belongs, and the API only needs its latitude and longitude. I used one sampled point in NO3 to download hourly ERA5 weather data for 2024.

I used daily average temperature to compare two ways of finding unusual observations. For SPC, I removed slow seasonal changes with the DCT before calculating the limits. It took me a while to understand why I needed both the median and a robust estimate of standard deviation. The median tells me where to measure from, and the spread tells me how far away a value needs to be before I flag it. LOF looked at the same temperatures differently: it flagged values that stood out compared with similar temperatures across the year.

I also used STL and a spectrogram on the weekly reservoir filling levels for NO3. Trying different trend settings helped me see that a shorter window lets the trend follow more of the changes, leaving less variation in the residuals. The spectrogram took me longer to read. What helped was thinking of a cycle as one rise and fall in the filling level. The brighter colours near the bottom of my plot showed that slower changes were stronger than faster ones.

In Streamlit, I worked on giving the reservoir and weather analyses their own pages, with two tabs on each. Reusing the NVE data loader made the reservoir page simpler to set up. I also revisited the electricity transfer page from Part 2 because the controls and charts felt crowded. That made me think more about how to arrange the page so someone can understand the plots without having read my notebook.

AI Usage

I used AI assistance from ChatGPT and GitHub Copilot in VS Code throughout Part 3. I used ChatGPT to understand the analysis methods (DCT and robust statistics, LOF, STL, and spectrograms), interpret the plots, and adapt code from the notebook for the Streamlit pages. I used GitHub Copilot for code suggestions in VS Code, which I adjusted to fit my project. I also used ChatGPT to help draft and edit the project log.

GitHub and Streamlit app

GitHub repository: malkia777/IND320---Project

Streamlit app: https://ind320---project-na9lbm8ud4jhtvejiciia6.streamlit.app/